

Pipeline Dữ Liệu Taxi NYC

Ngôn ngữ: [Tiếng Việt](#) (file này) · [English](#)

Nền tảng kỹ thuật dữ liệu đầu-cuối: ingest batch + streaming + CDC từ bản ghi Yellow Taxi NYC TLC (Parquet) và PostgreSQL CDC, biến đổi qua Spark → Trino → dbt → Superset, điều phối bằng Airflow trên Kubernetes (kind + Skaffold + Helm).

Pipeline ingest bản ghi chuyển đi Yellow Taxi NYC từ hai nguồn, validate và enrich, rồi cung cấp kết quả cho ba nhóm đối tượng:

Đối tượng	Sản phẩm	Cách dùng
Data engineers	33 gold dataset Parquet ở <code>s3a://nyc-gold/</code>	Xây pipeline mới, train model, audit, export
BI / marketing / sales / CEO	30 chart Superset + dashboard trên Postgres Analytics	Quyết định vận hành và chiến lược
Data engineers / SRE	Trino views, dbt marts, Airflow DAGs	Ad-hoc SQL, dbt lineage, debug pipeline

Hai đường ingest

- Batch** (`jobs/spark_local_batch.py`) — backfill các tháng lịch sử từ Parquet ở `s3a://nyc-raw/yellow_taxi/year=YYYY/month=MM/` . Hỗ trợ `--incremental` để chỉ xử lý các partition mới hơn partition đã có trong silver.

- **Streaming** (`jobs/spark_stream_taxi_events.py`) — consume `taxi.trip.events` từ Kafka qua `foreachBatch` , áp cùng logic enrich + validate như batch, append vào cùng đường dẫn silver.

Đường CDC (tùy chọn)

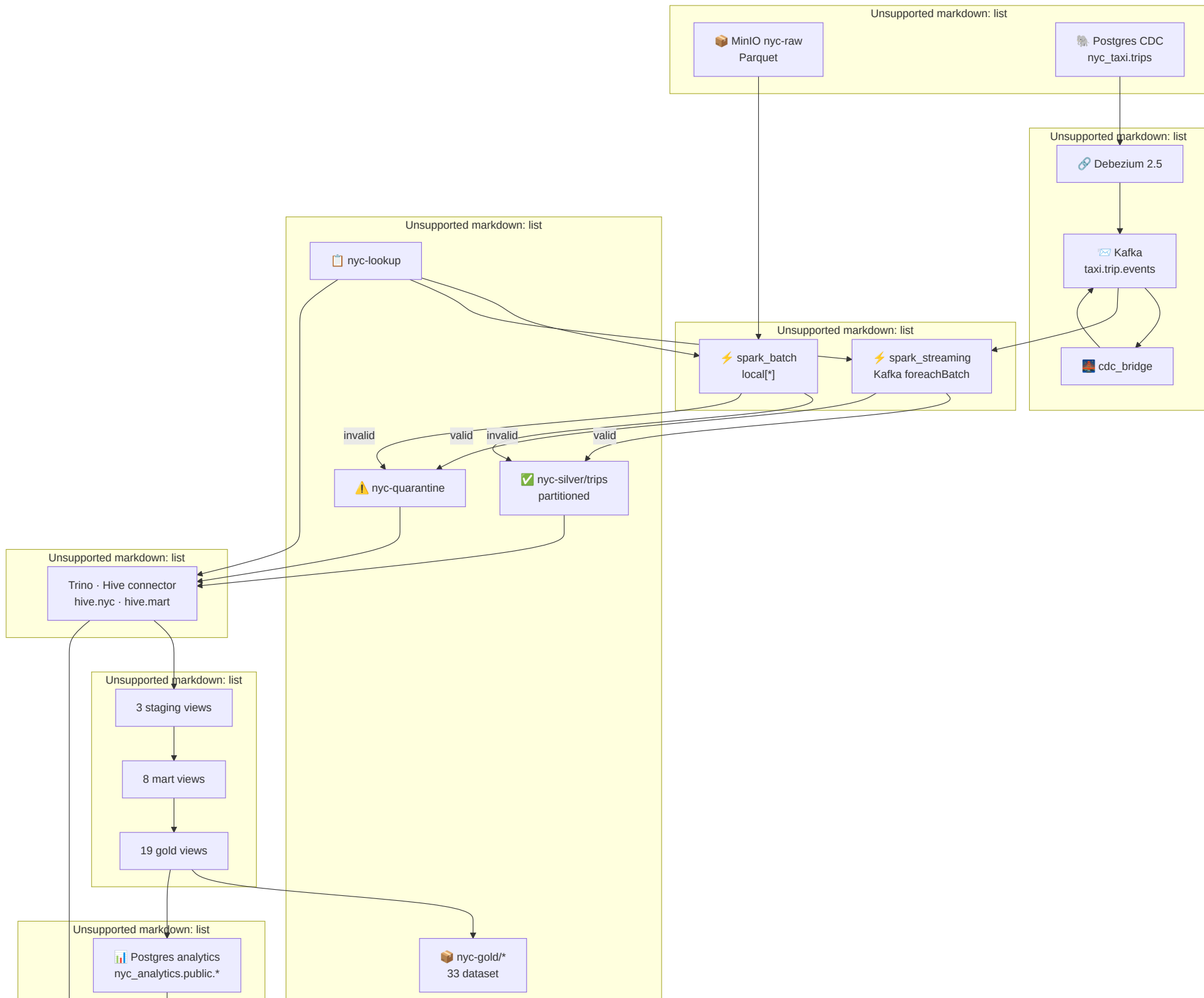
`Postgres 16 (WAL logical)` → `Debezium 2.5` → Kafka topic
`nyc_cdc.public.trips` → `scripts/cdc_bridge.py` → Kafka topic
`taxi.trip.events` → Spark Streaming → silver. Ba task DAG độc lập
(`cdc_seed` → `cdc_register` → `cdc_bridge`) seed bảng Postgres từ file Parquet, đăng ký connector Debezium, và bridge sự kiện CDC về format taxi chuẩn.

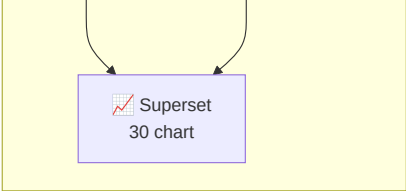
Sản phẩm đầu ra

- **Valid trips** → `s3a://nyc-silver/trips/` partition theo `pickup_year/pickup_month` (dataset hiện tại: ~8.4M dòng từ tháng 1–3/2024).
 - **Invalid trips** → `s3a://nyc-quarantine/invalid_trips/` (không partition, có `validation_errors` ARRAY<VARCHAR> và `quarantine_ts`).
 - **dbt views** → `hive.nyc.*` (raw external), `hive.mart.*` (dbt marts), expose qua Trino bằng Hive connector.
 - **Gold Parquet** → `s3a://nyc-gold/<dataset>/` — 33 dataset từ `scripts/export_gold_to_minio.py` (CTAS).
 - **Postgres analytics** → `nyc_analytics.public.*` — atomic swap từ `scripts/materialize_to_postgres.py` .
 - **Superset** → 30 chart definitions + dashboard trên `nyc_analytics.public.*` .
-

2. Kiến trúc

2.1 Sơ đồ thành phần

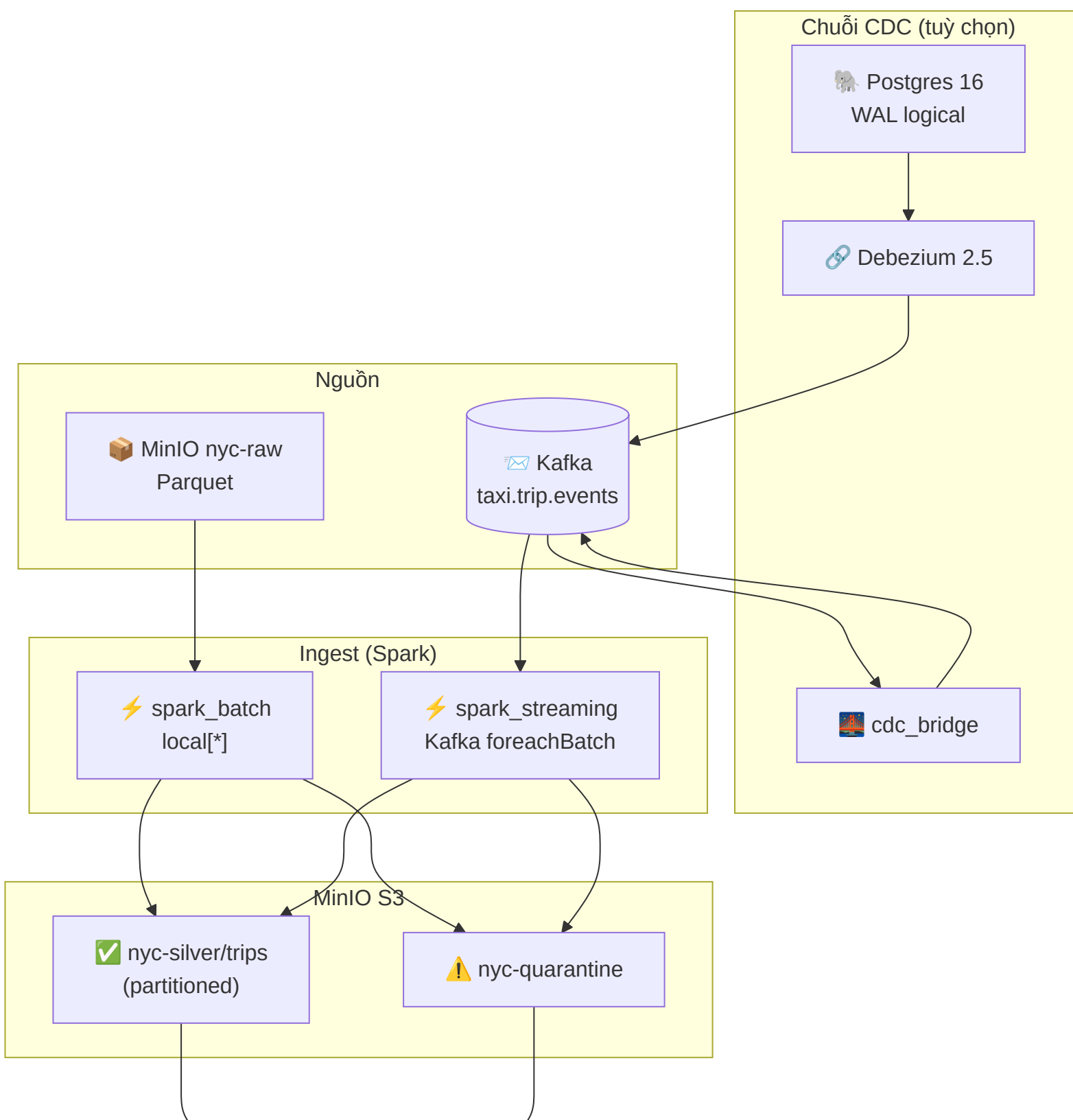


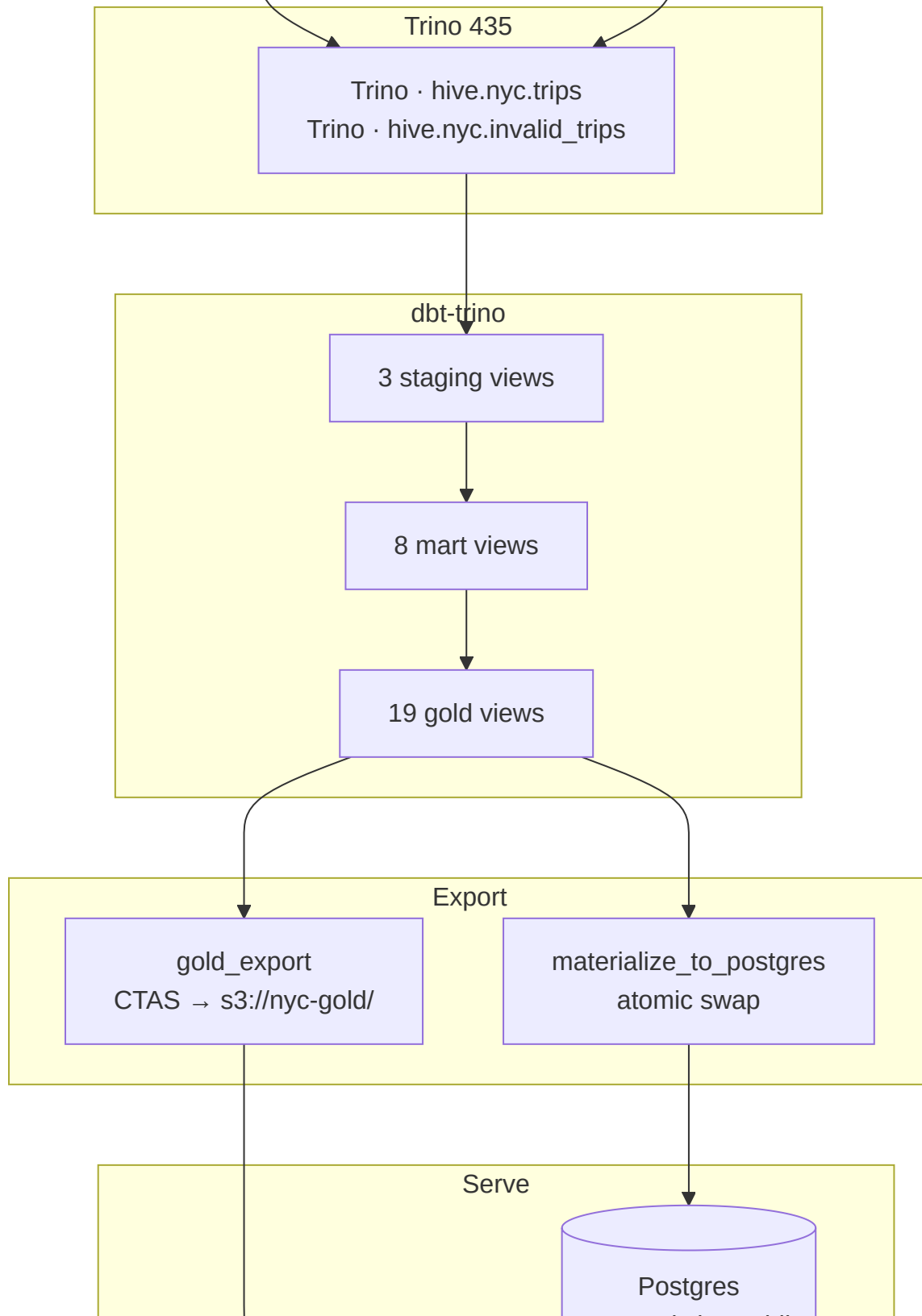


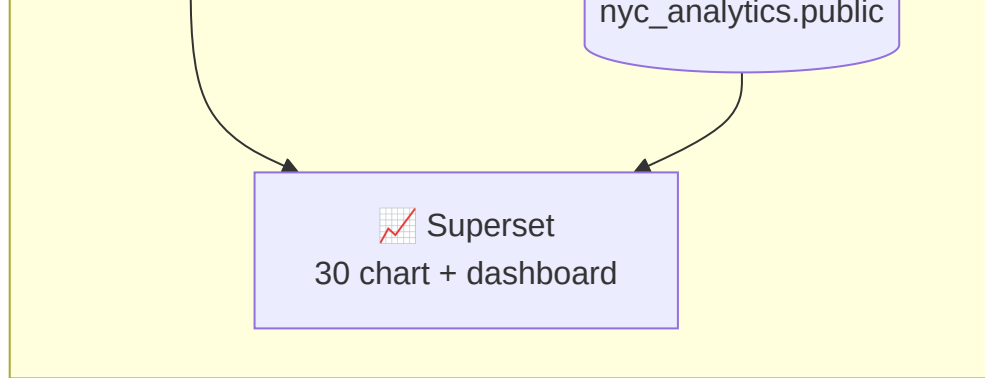
2.2 Tech stack

Tầng	Công nghệ	Phiên bản
Lưu trữ	MinIO S3	(latest)
Xử lý batch	Apache Spark	3.5.1
Xử lý stream	Apache Spark Structured Streaming	3.5.1
Nhắn tin	Confluent Kafka + ZooKeeper	7.6.1
CDC	Debezium Kafka Connect	2.5
SQL catalog	Trino (Hive connector, file-based metastore)	435
Biến đổi	dbt-trino	≥1.7, <2.0
Trực quan hoá	Apache Superset	4.0.0
Điều phối	Apache Airflow (KubernetesPodOperator)	2.10.5
Analytics store	PostgreSQL 16	16-alpine
Container / deploy	kind + Skaffold + Helm	kind ≥0.20, Skaffold v2, Helm ≥3
Alt: dev local	Docker Compose	(16 services)

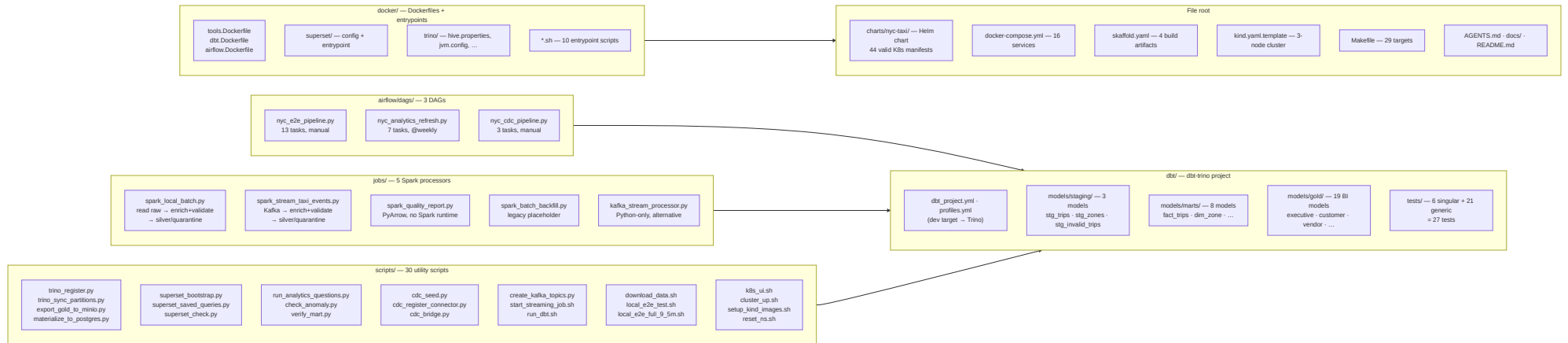
2.3 Luồng dữ liệu tổng quan



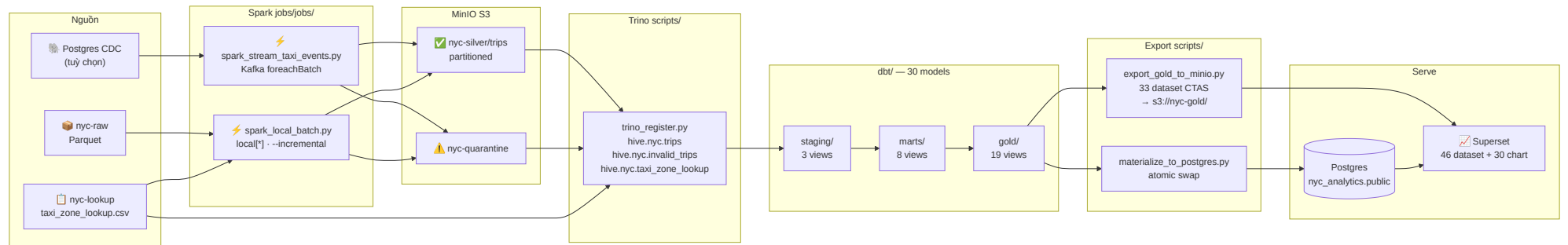


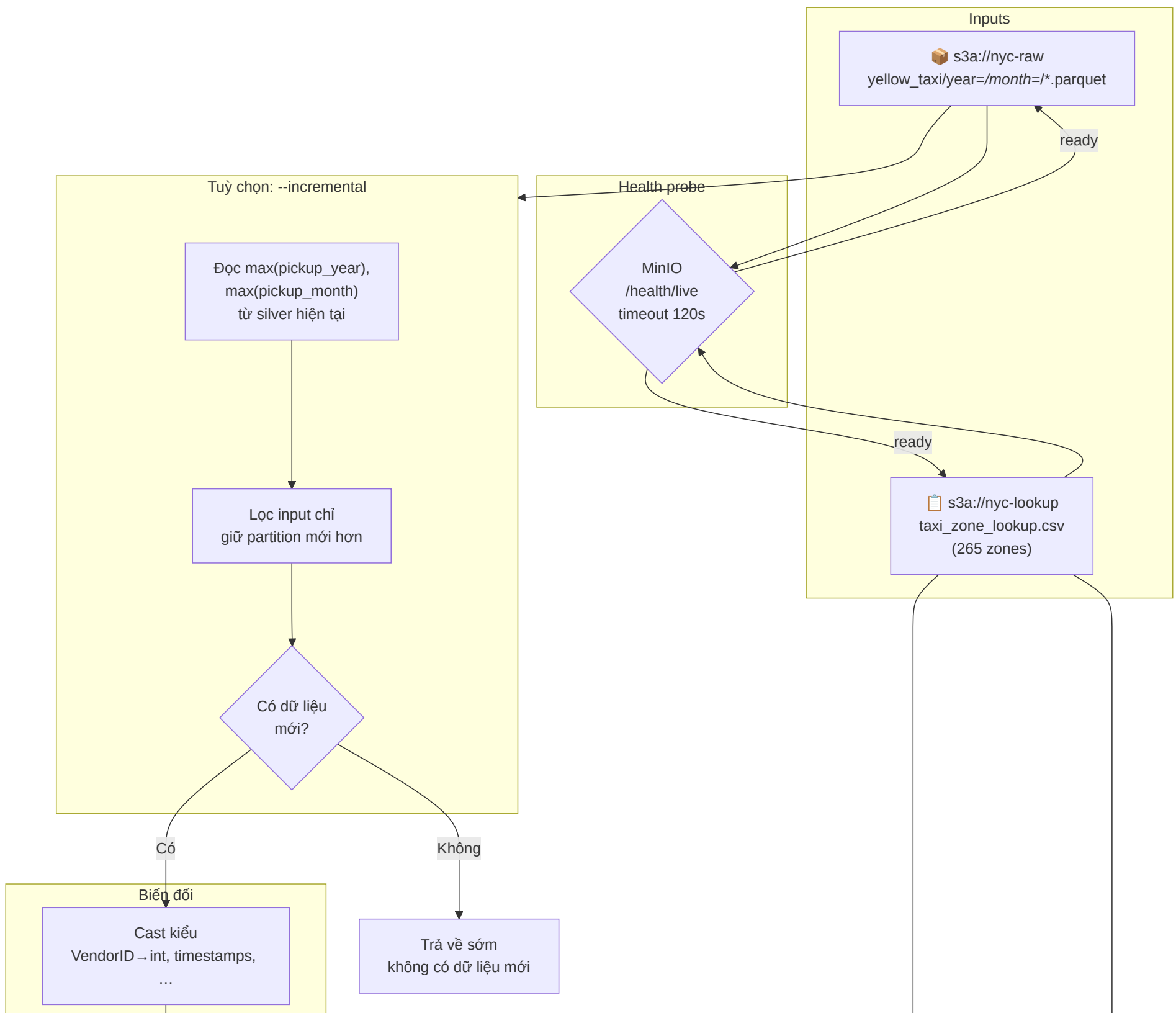


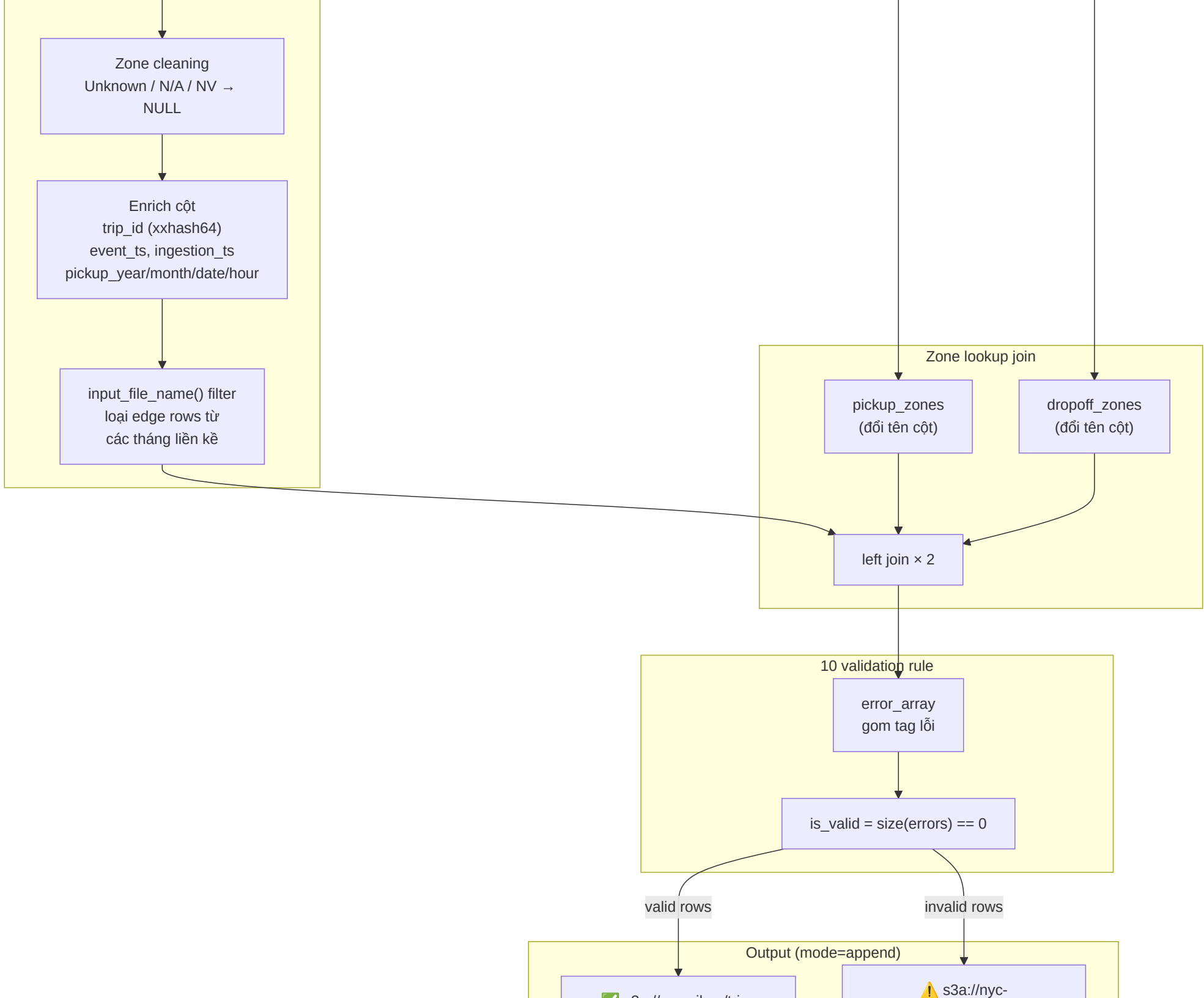
3. Cấu trúc repo



4. Thành phần chi tiết



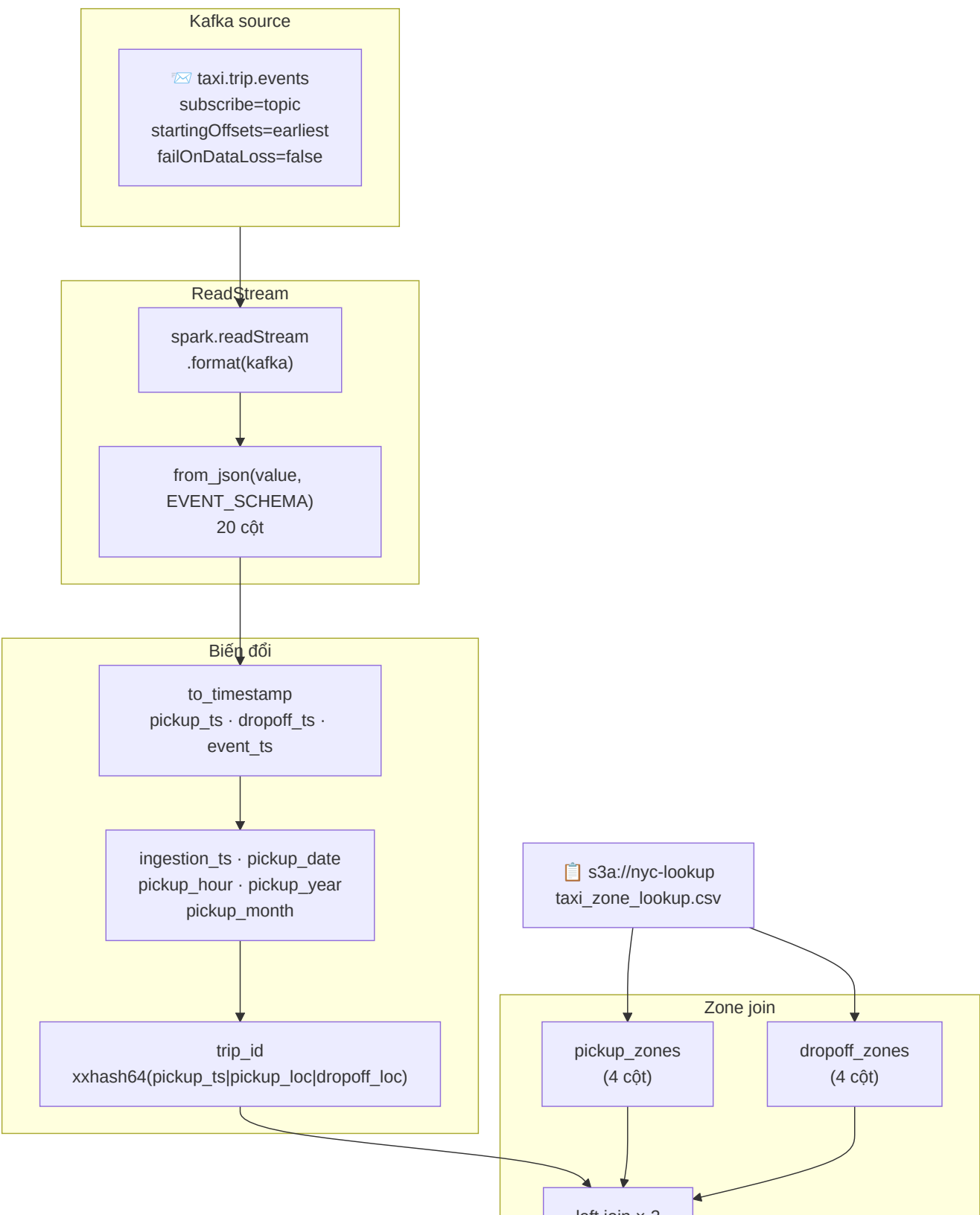


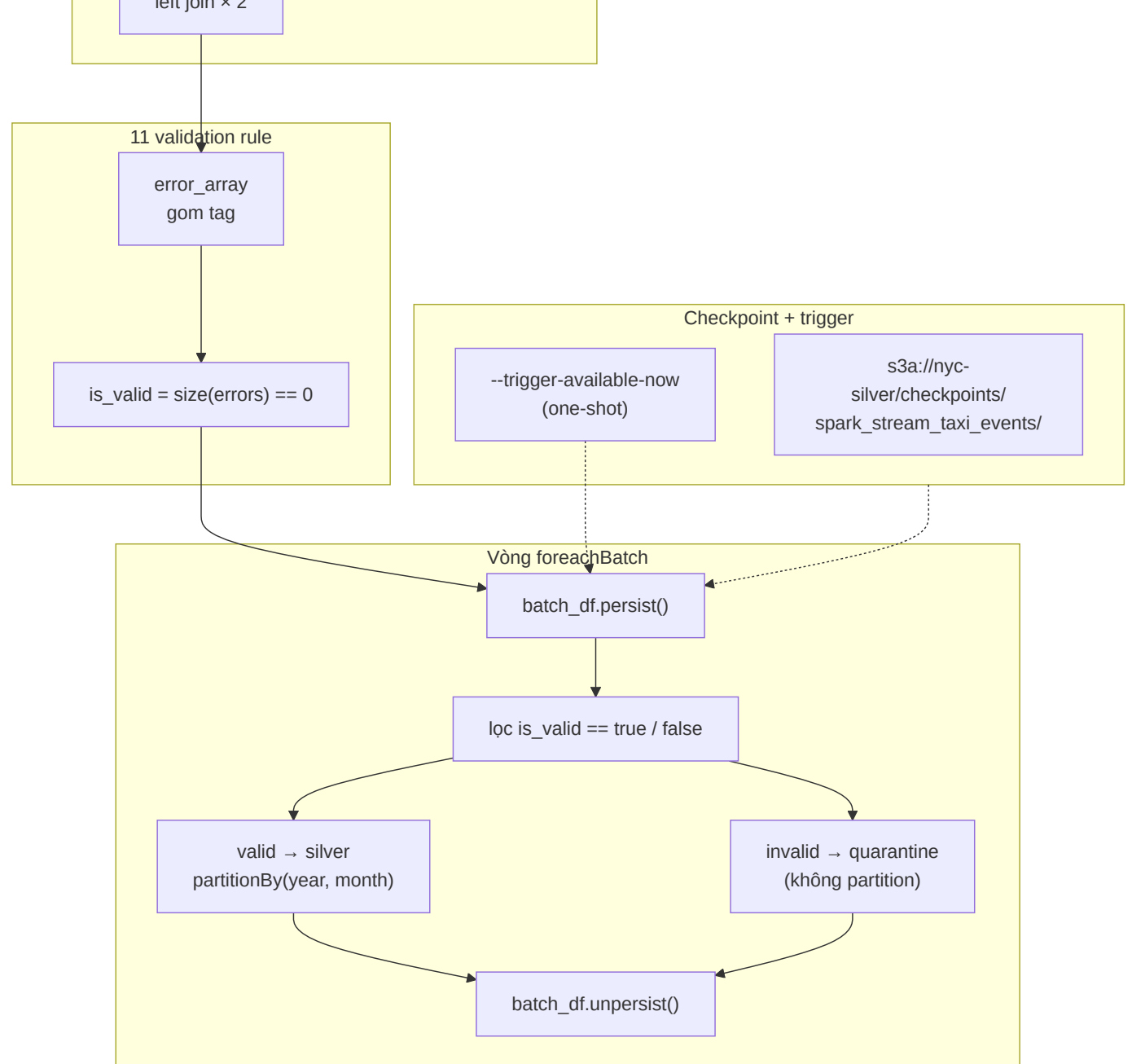


✓ s3a://nyc-silver/trips
partition theo
pickup_year/pickup_month

quarantine/invalid_trips
+ validation_errors
+ quarantine_ts

- **Master:** `local[*]` (một pod, chạy cả driver và executor).
- **Input:** `s3a://nyc-raw/yellow_taxi/year=*/month=*/*.parquet`.
- **Lookups:** `s3a://nyc-lookup/taxi_zone_lookup.csv` (265 dòng).
- **Output:** `s3a://nyc-silver/trips` (valid, partition theo `pickup_year/pickup_month`, `mode("append")`) và `s3a://nyc-quarantine/invalid_trips` (không partition).
- **CLI args:** `--input`, `--lookup`, `--silver`, `--quarantine`, `--year`, `--month`, `--incremental`.
- **S3A packages:** `org.apache.hadoop:hadoop-aws:3.3.4` + `com.amazonaws:aws-java-sdk-bundle:1.12.262` qua `--packages`.
- **MinIO health probe** trước khi đọc (`/minio/health/live`, 120s).
- **Chế độ Incremental:** đọc `max(pickup_year)`, `max(pickup_month)` từ đường dẫn silver hiện tại, lọc input chỉ giữ partition mới hơn. Trả về sớm nếu không có gì mới.
- **Year/month filter** qua `input_file_name()` — loại edge rows thuộc tháng liền kề ngay cả khi file được đặt tên cho tháng mục tiêu.
- **Zone cleaning:** `Borough.isin("Unknown", "N/A", "NV")` → `NULL` (xử lý NYC TLC zone ID 264 và 265).
- **Enriched columns:** `trip_id` (xxhash64 của `pickup_ts|pickup_loc|dropoff_loc`), `event_ts`, `ingestion_ts`, `pickup_date`, `pickup_hour`, `pickup_year`, `pickup_month`, cộng với 6 cột zone × 2 (pickup + dropoff).
- **10 validation rule** (xem §4.5).

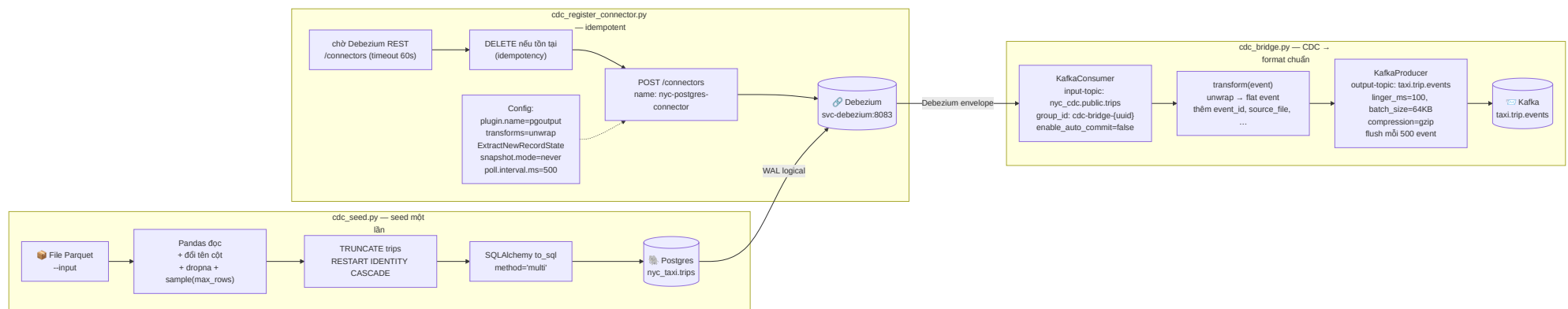




- **Master:** `local[*]` (one-shot qua `--trigger-available-now`, nếu không thì chạy liên tục).
- **Kafka:** `subscribe=taxi.trip.events`, `startingOffsets=earliest`, `failOnDataLoss=false`.

- **Schema:** 20 cột khai báo trong `EVENT_SCHEMA` (`vendor_id`, `pickup_datetime`, `dropoff_datetime`, `passenger_count`, `trip_distance`, `payment_type`, ...).
- **Đường dẫn output:** giống batch (`s3a://nyc-silver/trips` và `s3a://nyc-quarantine/invalid_trips`).
- **Checkpoint:** `s3a://nyc-silver/checkpoints/spark_stream_taxi_events/taxi.trip.events` .
- **foreachBatch** : persist batch DataFrame, tách valid/invalid, ghi từng phần, unpersist.
- **11 validation rule:** 10 từ batch + `event_id_null` (chỉ streaming).
- **CLI args:** `--bootstrap-server` , `--topic` , `--lookup-path` , `--silver-path` , `--quarantine-path` , `--checkpoint-path` , `--trigger-available-now` .

4.3 CDC scripts



- `scripts/cdc_seed.py` — đọc file Parquet bằng Pandas, đổi tên cột, TRUNCATE `nyc_taxi.trips` , insert qua `to_sql(method='multi')` . Mặc định `--max-rows 5000` , sample `random_state=42` .
- `scripts/cdc_register_connector.py` — POST config Debezium connector lên `${debezium-url}/connectors` . Idempotent (DELETE rồi POST). Cấu

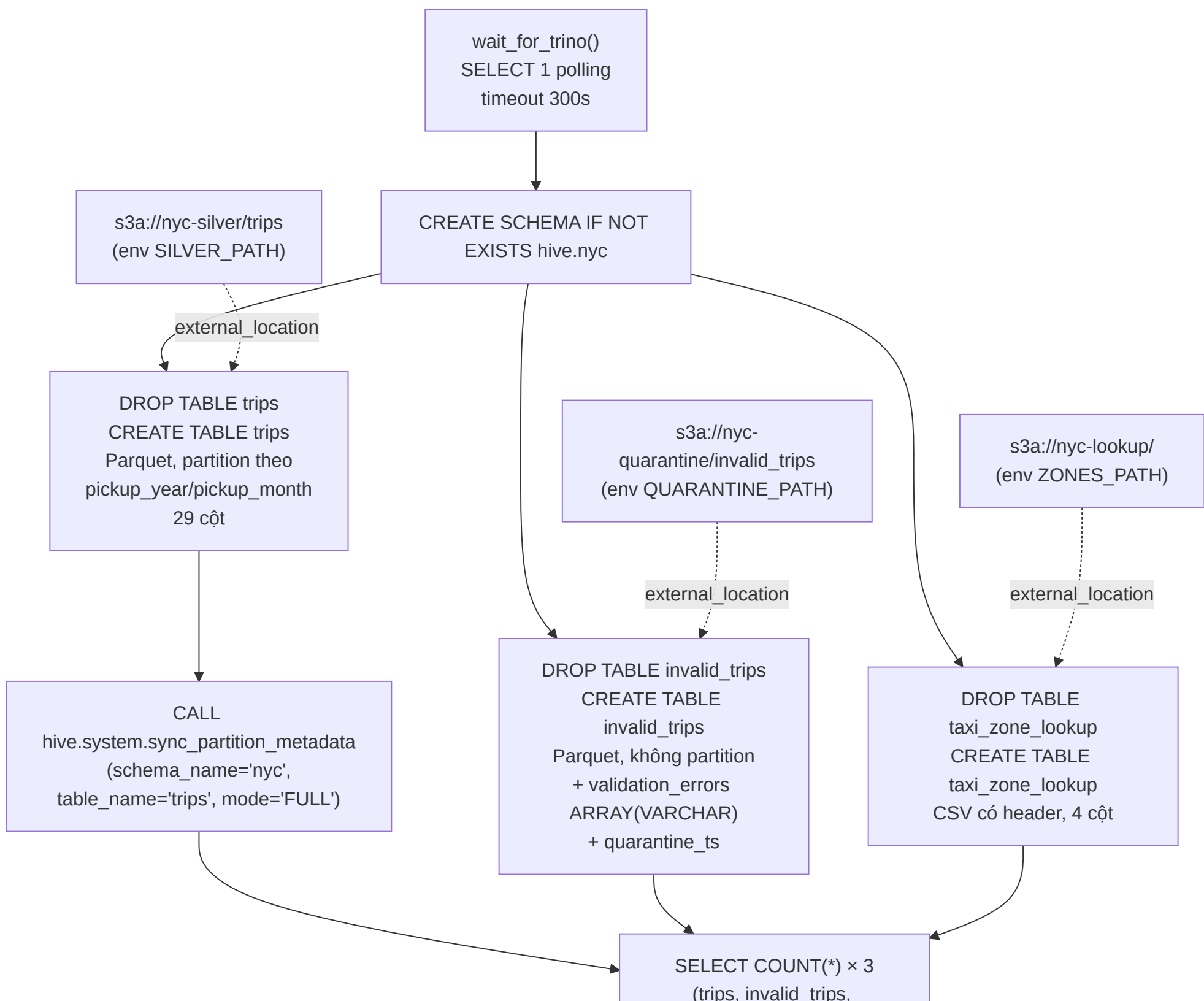
hình chính: `plugin.name=pgoutput` , `transforms=unwrap` với

`ExtractNewRecordState` , `snapshot.mode=never` ,

`transforms.unwrap.drop.tombstones=false` , `poll.interval.ms=500` .

- `scripts/cdc_bridge.py` — consume topic Debezium, transform mỗi event về format NYC Taxi chuẩn (cùng field với streaming schema), produce sang `taxi.trip.events` . Hiệu năng: `linger_ms=100` , `batch_size=65536` , `compression_type=gzip` . Async send với flush định kỳ mỗi `--flush-interval` event. **Group id sinh mỗi run** (`cdc-bridge-{uuid4.hex[:8]}`), nên mỗi lần chạy đọc từ offset sớm nhất — chấp nhận được cho one-shot DAG nhưng gây trùng event nếu bridge bị restart trong khi topic còn data chưa consume. Exit code: `0` khi tắt graceful, `1` khi lỗi kết nối Kafka.

4.4 Trino catalog — `scripts/trino_register.py`



(`pickup_year`, `pickup_month`),
`taxi_zone_lookup`)

- Idempotent: `DROP TABLE IF EXISTS` + `CREATE TABLE` cho 3 bảng.
- `hive.nyc.trips` (Parquet, partition theo `pickup_year/pickup_month`).
- `hive.nyc.invalid_trips` (Parquet, không partition, có `validation_errors ARRAY(VARCHAR)` và `quarantine_ts`).
- `hive.nyc.taxi_zone_lookup` (CSV có header).
- `CALL hive.system.sync_partition_metadata(schema_name => 'nyc', table_name => 'trips', mode => 'FULL')` sau khi tạo.
- Smoke test: `SELECT COUNT(*)` cho cả 3 bảng.
- Đường dẫn mặc định đọc từ env `SILVER_PATH`, `QUARANTINE_PATH`, `ZONES_PATH` (override được cho S3 vs local FS).

4.5 Validation rule

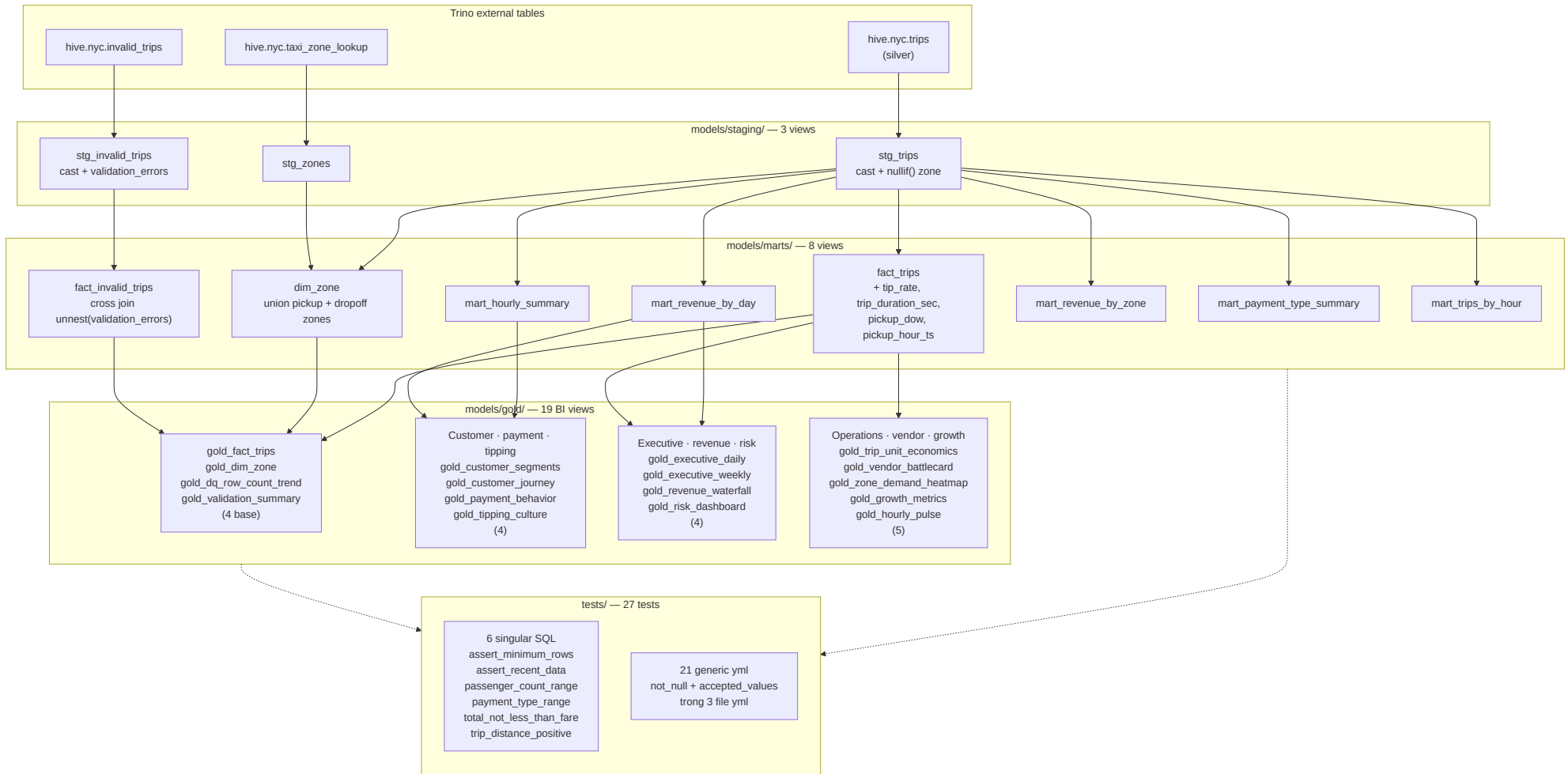
10 rule áp dụng trong batch; streaming thêm rule thứ 11 cho `event_id`.

Mỗi rule sinh ra một chuỗi tag trong mảng `validation_errors`; những dòng có mảng rỗng được ghi vào silver, phần còn lại vào quarantine.

#	Rule (chuỗi lỗi batch)	Rule (chuỗi lỗi streaming)
1	<code>pickup_datetime_null_or_invalid</code>	giống
2	<code>dropoff_datetime_null_or_invalid</code>	giống
3	<code>invalid_trip_duration</code>	giống
4	<code>non_positive_trip_distance</code>	<code>trip_distance_must_be_gt_0</code>
5	<code>negative_fare_amount</code>	<code>fare_amount_must_be_gte_0</code>
6	<code>total_amount_less_than_fare</code>	<code>total_amount_must_be_gte_fare_amount</code>
7	<code>invalid_passenger_count</code>	<code>passenger_count_out_of_range</code>

#	Rule (chuỗi lỗi batch)	Rule (chuỗi lỗi streaming)
8	payment_type_out_of_range	giống
9	unknown_pickup_location	pickup_location_not_found
10	unknown_dropoff_location	dropoff_location_not_found
11	(không có)	event_id_null (chỉ streaming)

Tầng dbt staging chạy thêm `nullif(nullif(...))` trên cột zone
(`Unknown` , `N/A` , `NV`) như lớp dọn dẹp dự phòng.



- 30 SQL model:

- **Staging (3)** — `stg_trips`, `stg_zones`, `stg_invalid_trips`. Tất cả `materialized='view'`. Cast kiểu và dọn chuỗi zone null-ish.
- **Marts (8)** — `fact_trips`, `fact_invalid_trips`, `dim_zone`, `mart_hourly_summary`, `mart_revenue_by_day`, `mart_revenue_by_zone`, `mart_payment_type_summary`, `mart_trips_by_hour`. Tất cả `materialized='view'`.
- **Gold (19)** — `gold_fact_trips`, `gold_dim_zone`, `gold_mart_revenue_by_day`, `gold_mart_revenue_by_zone`, `gold_dq_row_count_trend`, `gold_validation_summary`, cộng 13 model BI

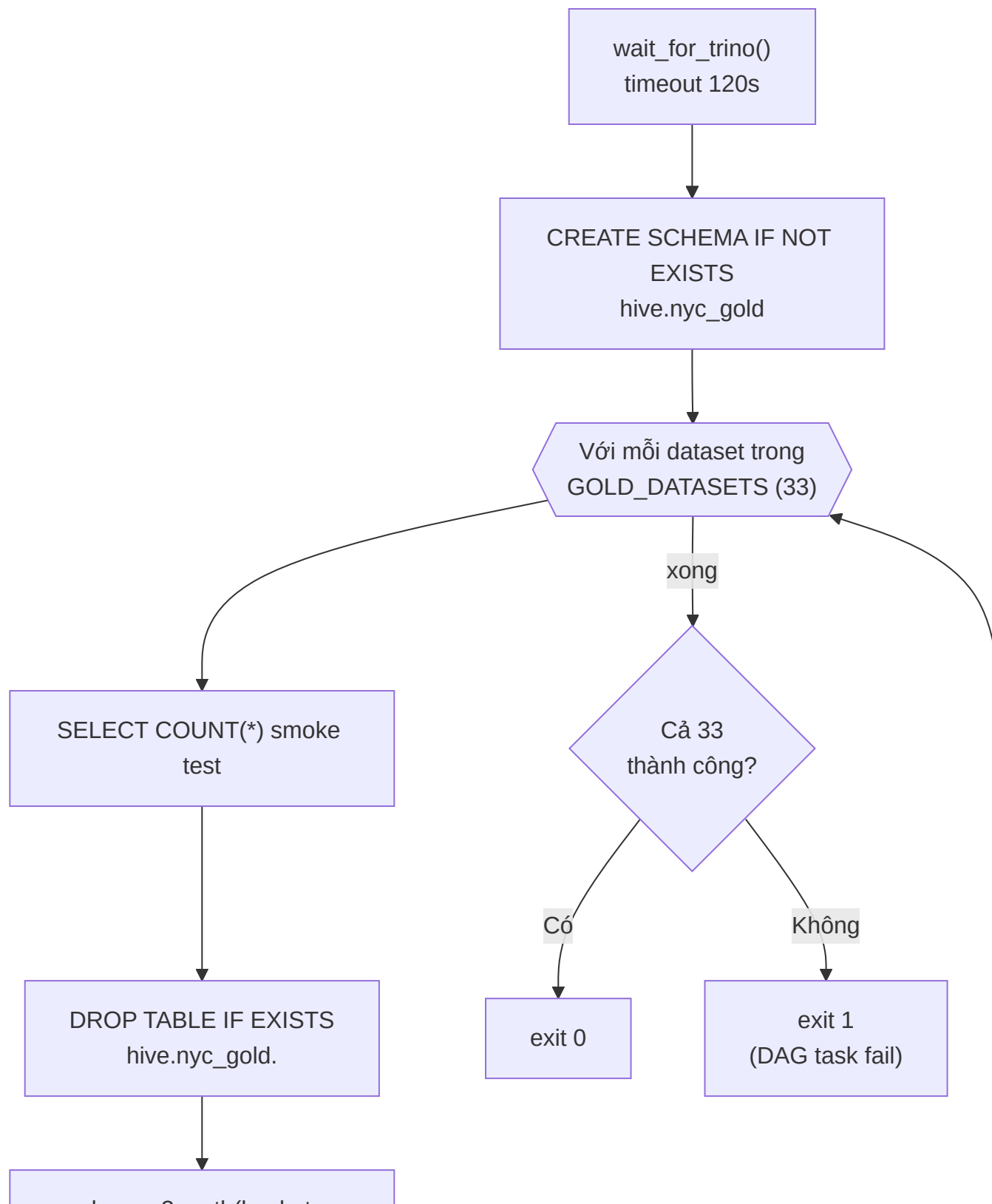
```
(gold_customer_segments, gold_customer_journey,  
gold_payment_behavior, gold_tipping_culture,  
gold_trip_unit_economics, gold_zone_demand_heatmap,  
gold_vendor_battlecard, gold_growth_metrics,  
gold_executive_daily, gold_executive_weekly,  
gold_revenue_waterfall, gold_hourly_pulse, gold_risk_dashboard).
```

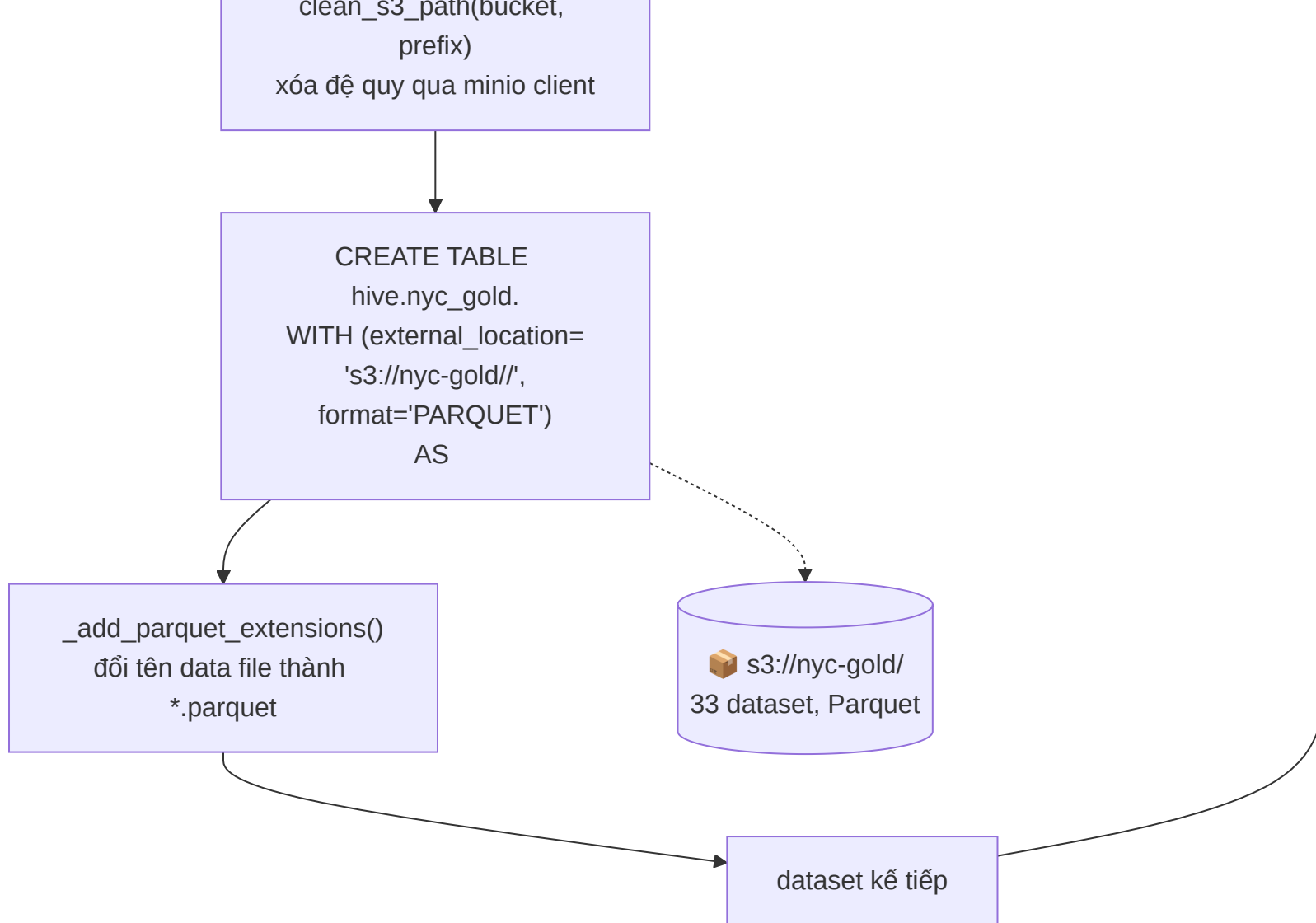
Tất cả `materialized='view'` trong dev.

- **27 test:**

- 6 singular SQL: `assert_minimum_rows`, `assert_recent_data`,
`passenger_count_range`, `payment_type_range`,
`total_not_less_than_fare`, `trip_distance_positive`.
- 21 generic trong file yml (`fact_invalid_trips_tests.yml`,
`fact_trips_tests.yml`, `stg_trips_tests.yml`) — kiểm tra `not_null`
và `accepted_values`.
- `profiles.yml` — target `dev` trở vào Trino (`type: trino`,
`host: svc-trino`, `port: 8080`, `database: hive`, `schema: mart`,
`threads: 4`).
- `dbt build` chạy models + test; kỳ vọng `30 models, 27 tests`.

4.7 Gold export — `scripts/export_gold_to_minio.py`





- 33 dataset định nghĩa trong list `GOLD_DATASETS`. Mỗi entry: `{name, sql, location_subdir}`.
- Với mỗi dataset: smoke test `SELECT COUNT(*)` → `DROP TABLE IF EXISTS hive.nyc_gold.<name>` → `clean_s3_path` (xóa đệ quy các object cũ trong prefix) → `CREATE TABLE ... WITH (external_location=...) AS <sql>`.
- Đổi tên file Parquet thành đuôi `.parquet` (đôi khi Hive ghi ra không có đuôi tùy layout partition).
- Trả exit 0 nếu tất cả thành công, 1 nếu có dataset lỗi.

wait_for_trino()
300s

wait_for_postgres()
psycopg2 connect, 120s

from export_gold_to_minio
import GOLD_DATASETS
(single source of truth —
33 SQL)

Với mỗi dataset (33)

xong

Cả 33
thành công?

Có

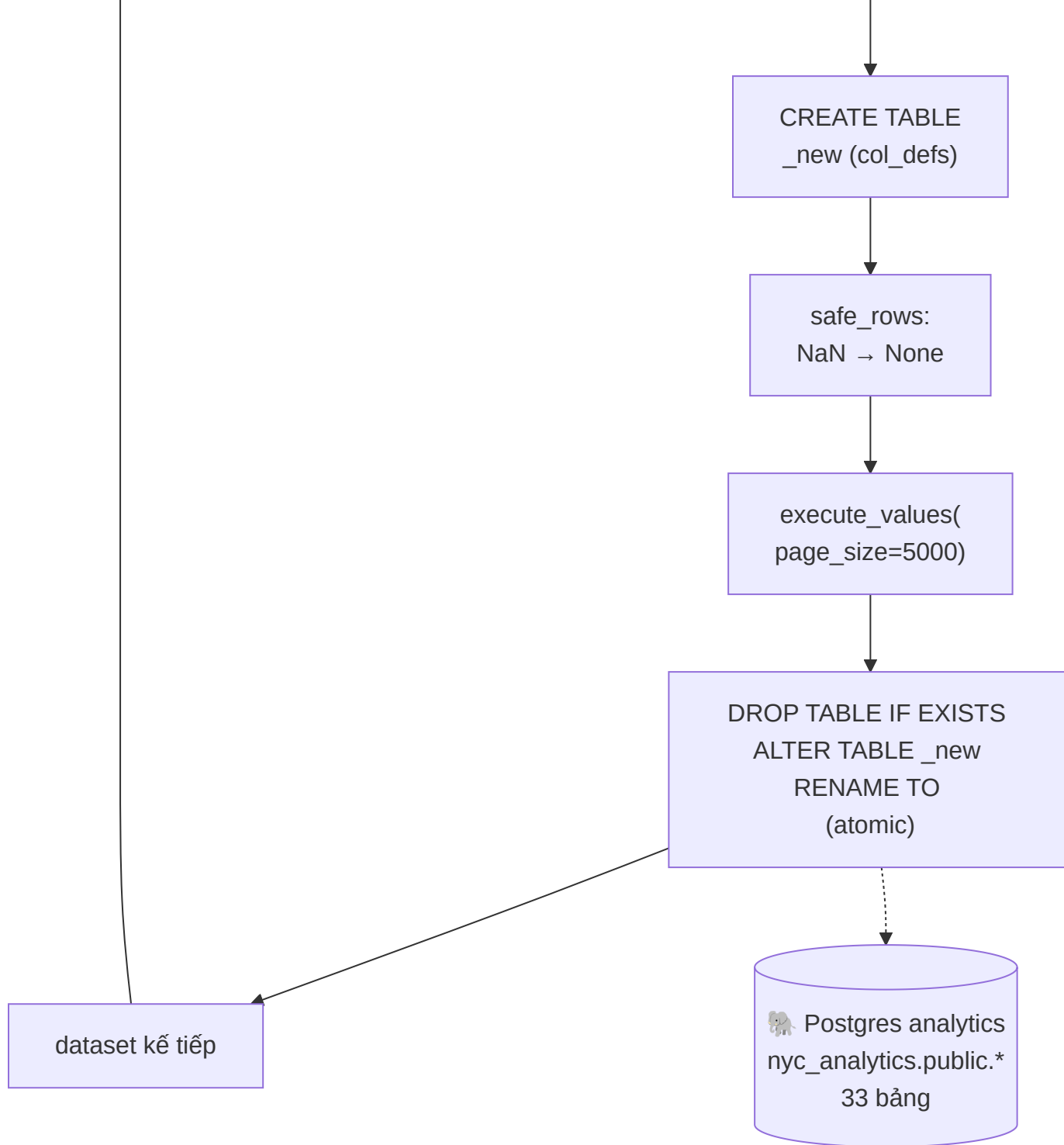
Không

exit 0

exit 1

trino_cur.execute(sql)
fetchall()

TYPE_MAP trino →
postgres
(varchar → TEXT,
double → DOUBLE
PRECISION, ...)

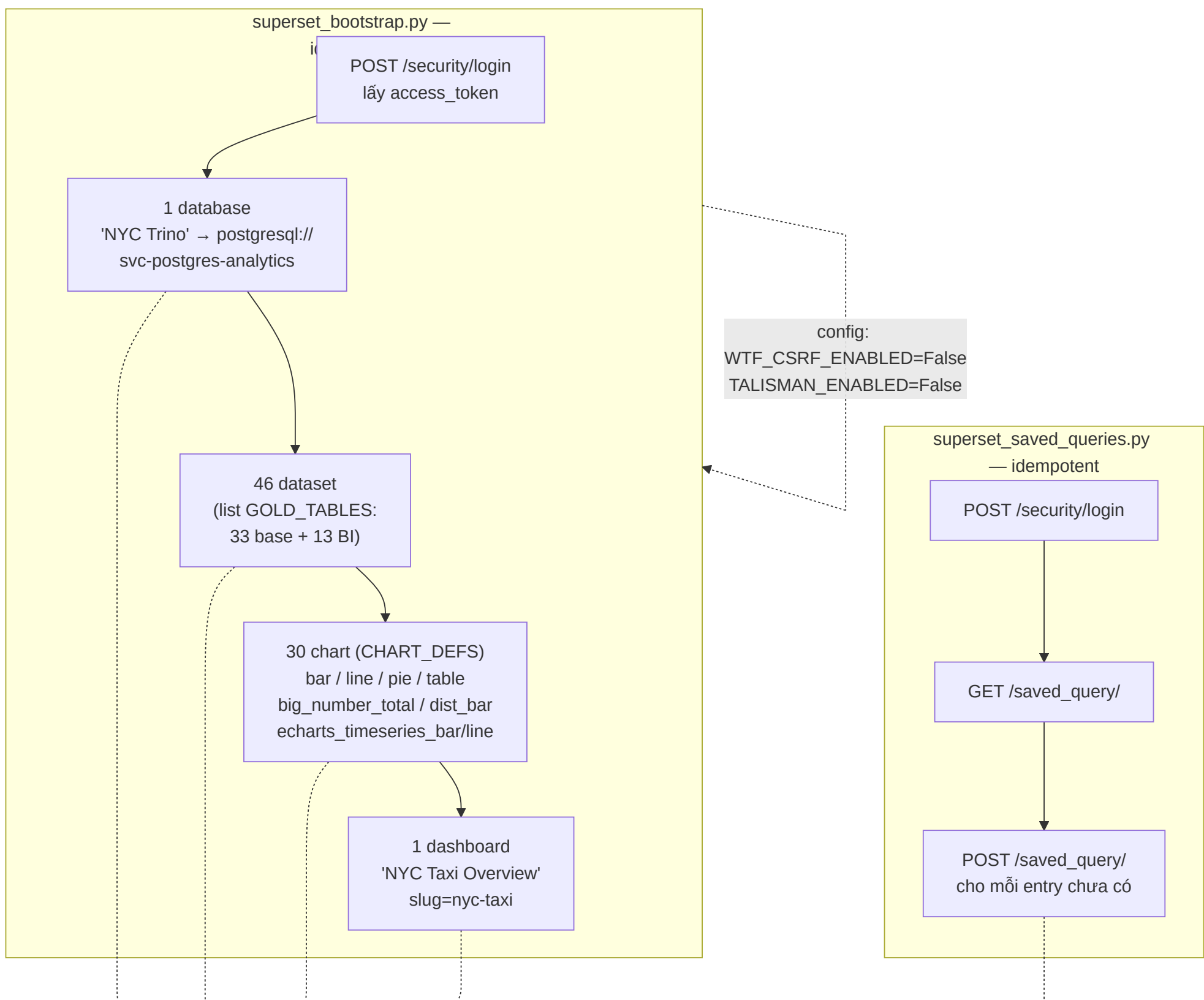


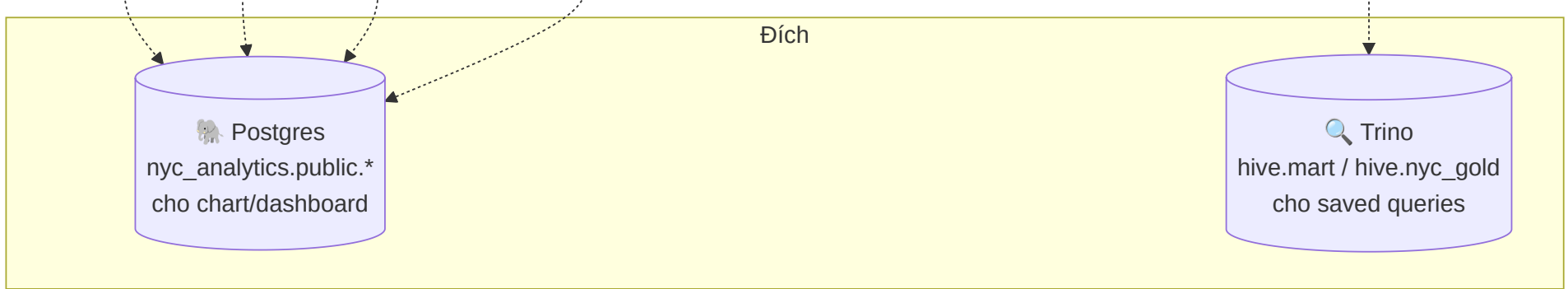
- Import `GOLD_DATASETS` từ `export_gold_to_minio.py` để SQL là single source of truth.

- Với mỗi dataset: chạy cùng SQL đó trên Trino, đọc kết quả, map kiểu Trino → Postgres (`TYPE_MAP`), tạo `<name>_new` , batch insert qua `execute_values(page_size=5000)` , rồi atomic-swap:
`DROP TABLE IF EXISTS <name>; ALTER TABLE <name>_new RENAME TO <name> .`
- Làm sạch `NaN` → `None` trước khi insert.
- Trả exit 0 nếu tất cả thành công, 1 nếu dataset nào lỗi.

4.9 Superset — `scripts/superset_bootstrap.py` và

`scripts/superset_saved_queries.py`





- `superset_bootstrap.py` đăng ký (idempotent):
 - 1 database (NYC Trino → `postgresql://analytics:analytics@svc-postgres-analytics:5432/nyc_analytics`).
 - 46 dataset trong list `GOLD_TABLES` (33 base + 13 BI).
 - 30 chart definition trong `CHART_DEFS` (bar / line / pie / table / big-number / echarts timeseries / dist bar).
 - 1 dashboard `nyc-taxi` .
 - Dùng `WTF_CSRF_ENABLED = False` và `TALISMAN_ENABLED = False` cho các POST call bootstrap.
- `superset_saved_queries.py` đăng ký 25 saved query trong SQL Lab, trở vào schema `hive.mart` và `hive.nyc_gold` của Trino.

4.10 Anomaly check — `scripts/check_anomaly.py`

- Đọc `hive.mart.gold_dq_row_count_trend` và đánh dấu những dòng có `anomaly_flag != 'NORMAL'` .
- In các anomaly và summary (`low` , `high` , `total_days`).
- **Exit code 0 luôn** — chỉ thông tin, không block DAG.

5. Airflow DAGs

Cả ba DAG dùng `KubernetesPodOperator` (KPO) với các default chung:

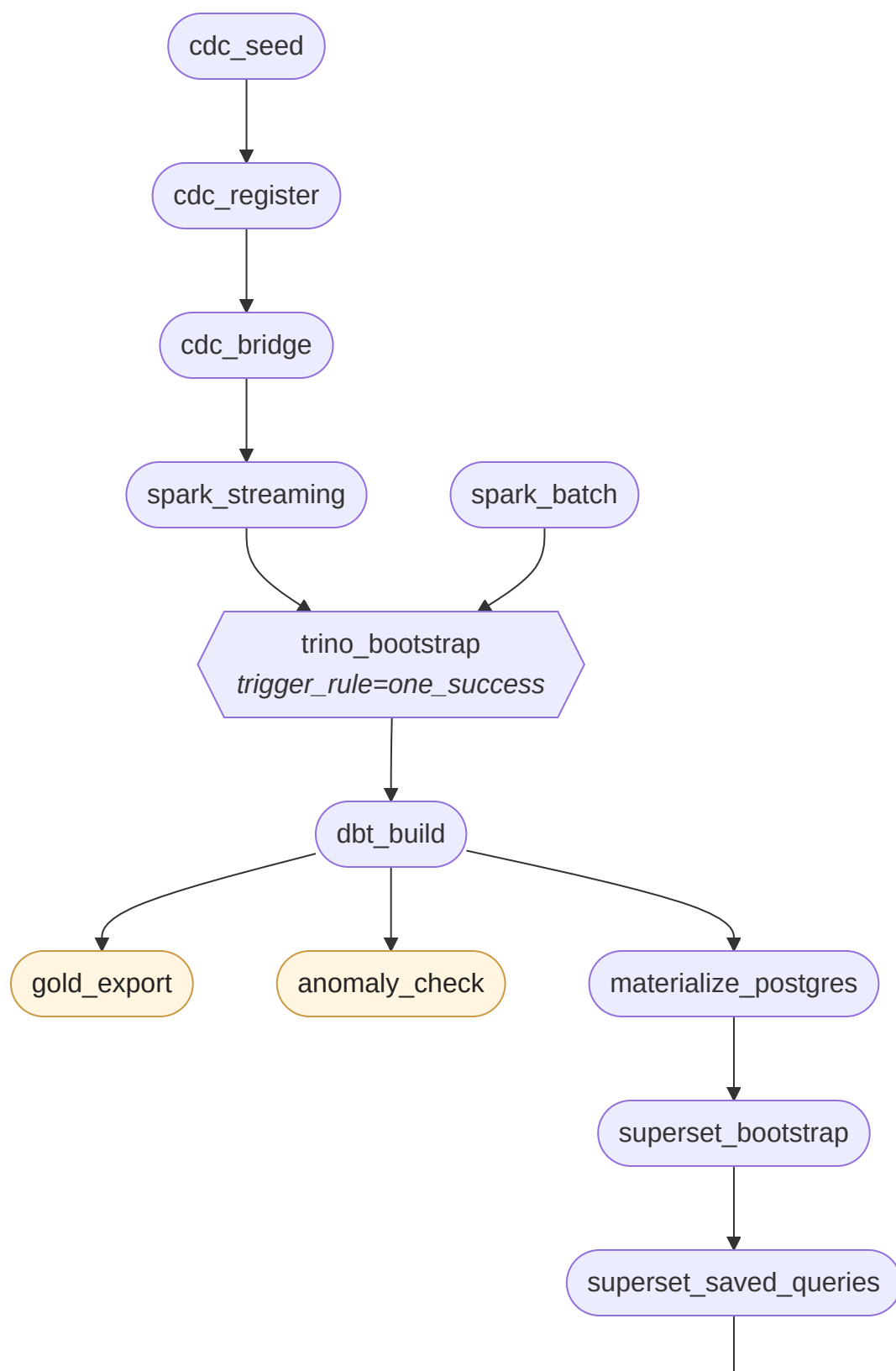
- `namespace="nyc-taxi"` , `in_cluster=True` , `service_account_name="airflow-sa"`
- `get_logs=True` (stream pod log vào Airflow task log)
- `volumes=[project_volume]` mount `project-files-pvc` vào `/opt/project` (chứa `airflow/dags/` , `jobs/` , `scripts/` , `dbt/` , `charts/`)
- `image_pull_policy="IfNotPresent"` (dùng image đã load vào kind local, không pull registry)
- `security_context=run_as_user=0` cho task Spark (để S3A filesystem có thể ghi vào PVC)

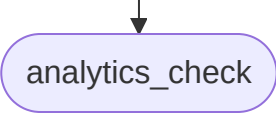
5.0 Cấu hình task chung

Thiết lập	nyc_e2e_pipeline	nyc_analytics_refresh	nyc_cdc_pipeline
retries	3	3	2
retry_delay	30s	30s	30s
execution_timeout	30 min	30 min	15 min
depends_on_past	False	False	False
max_active_runs	1	1	1
catchup	(default)	False	False
Schedule	None (manual)	@weekly	None (manual)
start_date	2024-01-01	2026-01-01	2024-01-01

5.1 `nyc_e2e_pipeline` — 13 task, trigger thủ công

Toàn bộ pipeline E2E: chuỗi CDC tùy chọn (seed → register → bridge) đẩy sự kiện vào đường streaming; batch và stream đều hội tụ tại Trino; dbt build view; gold + materialize chạy song song; Superset bootstrap lại; bộ câu hỏi analytics đóng run.





Trigger rule: `trino_bootstrap` dùng `trigger_rule="one_success"` — chạy nếu `spark_batch` HOẶC `spark_streaming` thành công. Cho phép chạy chỉ batch, chỉ CDC, hoặc cả hai mà không phải sửa DAG. Phần còn lại dùng default `all_success`.

Giữ pod: `trino_bootstrap` và `cdc_seed` set `is_delete_operator_pod=False` để pod log còn lại sau khi task xong (hữu ích debug S3A / Trino bootstrap fail). 11 task còn lại dùng default `True` và tự cleanup.

`trino_bootstrap` startup: `startup_timeout_seconds=600` — Trino có thể cần khá lâu để ready lần đầu sau Helm deploy mới.

Chi tiết task

Task	Image	Command / Args
<code>cdc_seed</code>	<code>nyc-pipeline-tools:latest</code>	<code>entrypoint-cdc-seed --input /opt/project/data/raw/yellow_taxi/year=2024/month=01/yellow_tripdata_2024-01.parquet --max-rows 1000 --dsn postgresql://postgres:postgres@svc-postgres-cdc:5432/nyc_taxi</code>
<code>cdc_register</code>	<code>nyc-pipeline-tools:latest</code>	<code>entrypoint-cdc-register --debezium-url http://svc-debezium:8083 --postgres-host svc-postgres-cdc</code>
<code>cdc_bridge</code>	<code>nyc-pipeline-tools:latest</code>	<code>entrypoint-cdc-bridge --bootstrap-server svc-kafka:9092 --input-topic nyc_cdc.public.trips --output-topic taxi.trip.events --idle-timeout 30 --flush-interval 500</code>
<code>spark_batch</code>	<code>apache/spark:3.5.1</code>	<code>spark-submit --master local[*] --packages hadoop-aws:3.3.4,aws-java-sdk-bundle:1.12.262 --conf spark.jars.ivy=/opt/project/.ivy2 --conf</code>

Task	Image	Command / Args
		<pre>spark.hadoop.mapreduce.fileoutputcommitter.algorithm.version=2 --conf spark.scheduler.mode=FAIR /opt/project/jobs/spark_local_batch.py --input s3a://nyc-raw/yellow_taxi/year=*/month=*/*.parquet --lookup s3a://nyc-lookup/taxi_zone_lookup.csv --silver s3a://nyc-silver/trips --quarantine s3a://nyc-quarantine/invalid_trips --incremental</pre>
spark_streaming	apache/spark:3.5.1	<pre>spark-submit --master local[*] --conf spark.jars.ivy=/opt/project/.ivy2.conf spark.hadoop.mapreduce.fileoutputcommitter.algorithm.version=2 --conf spark.scheduler.mode=FAIR --packages spark-sql-kafka-0-10_2.12:3.5.1,hadoop-aws:3.3.4,aws-java-sdk-bundle:1.12.262 /opt/project/jobs/spark_stream_taxi_events.py --bootstrap-server svc-kafka:9092 --topic taxi.trip.events --lookup-path s3a://nyc-lookup/taxi_zone_lookup.csv --silver-path s3a://nyc-silver/trips --quarantine-path s3a://nyc-quarantine/invalid_trips --checkpoint-path s3a://nyc-silver/checkpoints/spark_stream_taxi_events/taxi.trip.events trigger-available-now</pre>
trino_bootstrap	nyc-pipeline-tools:latest	entrypoint-trino-bootstrap
dbt_build	nyc-dbt:latest	entrypoint-dbt
gold_export	nyc-pipeline-tools:latest	python3 /opt/project/scripts/export_gold_to_minio.py
materialize_postgres	nyc-pipeline-tools:latest	python3 /opt/project/scripts/materialize_to_postgres.py
superset_bootstrap	nyc-pipeline-tools:latest	python3 /opt/project/scripts/superset_bootstrap.py

Task	Image	Command / Args
superset_saved_queries	nyc-pipeline-tools:latest	python3 /opt/project/scripts/superset_saved_queries.py
analytics_check	nyc-pipeline-tools:latest	python3 /opt/project/scripts/run_analytics_questions.py
anomaly_check	nyc-pipeline-tools:latest	python3 /opt/project/scripts/check_anomaly.py

Ghi chú về image override: khi chạy DAG, flag `--incremental` trên

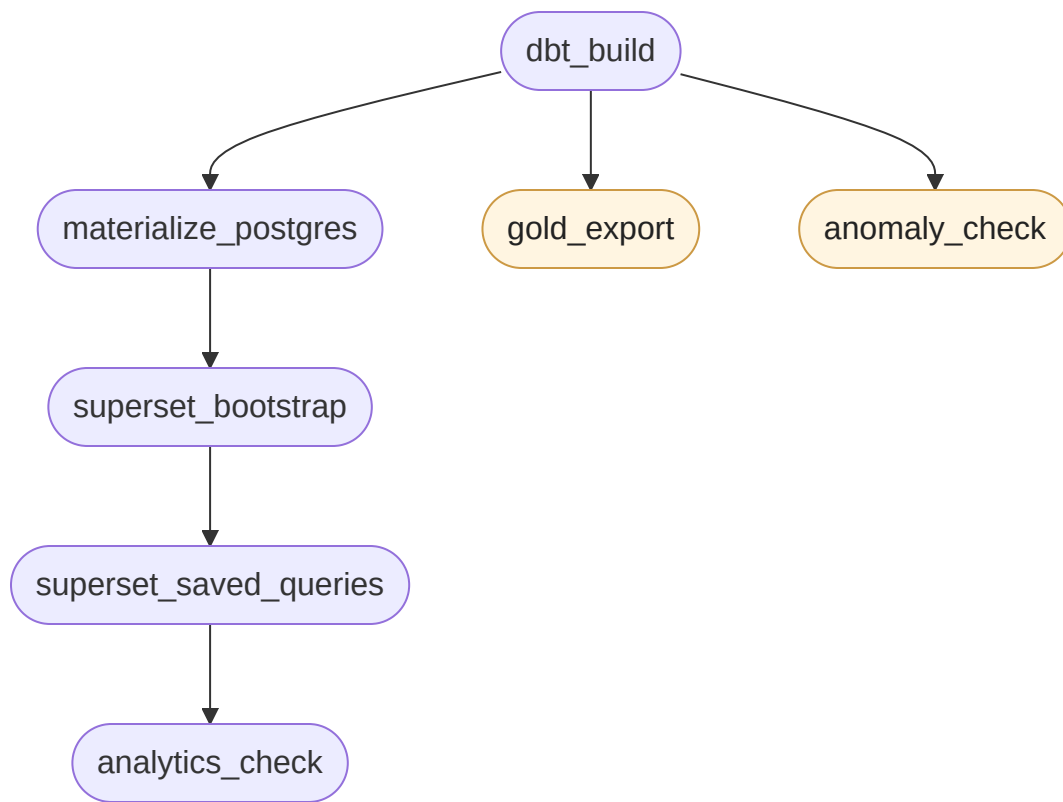
`spark_batch` có nghĩa nó sẽ skip partition nếu `max(pickup_year), max(pickup_month)` trong silver đã bao phủ file input.

Nhờ đó DAG có thể

trigger lại an toàn mà không trùng dòng.

5.2 `nyc_analytics_refresh` — 7 task, `@weekly`

Chạy lại tầng analytics với giả định tầng silver đã có data (không có task Spark — giả định đã chạy qua E2E DAG hoặc thủ công). Tái sử dụng cùng script và image pattern với E2E DAG.



Ngữ nghĩa fan-out: `dbt_build` có 3 task downstream:

- `materialize_postgres` — khởi động chuỗi materialize.
- `gold_export` — chạy song song, không có downstream (chỉ ghi s3://nyc-gold/).
- `anomaly_check` — chạy song song, không có downstream.

`gold_export` và `anomaly_check` là **đầu cuối nhánh**: chúng không block nhau và không block chuỗi `materialize_postgres`. Nghĩa là `gold_export` fail KHÔNG làm `analytics_check` bị skip; DAG chỉ fail đường analytics nếu `materialize_postgres` hoặc một trong các task Superset fail.

Chi tiết task (tất cả KPO, dùng cùng env var như §5.1 trừ khi override):

Task	Image	Command / Args	Ghi chú
dbt_build	nyc-dbt:latest	entrypoint-dbt	Giống §5.1
materialize_postgres	nyc-pipeline-tools:latest	python3 /opt/project/scripts/materialize_to_postgres.py	Giống §5.1
gold_export	nyc-pipeline-tools:latest	python3 /opt/project/scripts/export_gold_to_minio.py	Giống §5.1
anomaly_check	nyc-pipeline-tools:latest	python3 /opt/project/scripts/check_anomaly.py	Giống §5.1
superset_bootstrap	nyc-pipeline-tools:latest	python3 /opt/project/scripts/superset_bootstrap.py	Giống §5.1
superset_saved_queries	nyc-pipeline-tools:latest	python3 /opt/project/scripts/superset_saved_queries.py	Giống §5.1
analytics_check	nyc-pipeline-tools:latest	python3 /opt/project/scripts/run_analytics_questions.py	Giống §5.1

Giữ pod: khác §5.1, không task nào trong DAG này set `is_delete_operator_pod=False` . Log bị xoá khi task xong theo default.

5.3 `nyc_cdc_pipeline` — 3 task, thủ công

Dùng để demo chuỗi CDC end-to-end mà không chạy full E2E. Seed Postgres với 5000 dòng từ file Parquet, đăng ký Debezium connector, sau đó bridge sự kiện CDC vào topic taxi event chuẩn trong ~30 giây rồi thoát.



Chi tiết task:

Task	Image	Command / Args	Ghi chú
cdc_seed	nyc-pipeline-tools:latest	entrypoint-cdc-seed --input /opt/project/data/raw/yellow_taxi/year=2024/month=01/yellow_tripdata_2024-01.parquet --max-rows 5000 --dsn postgresql://postgres:postgres@svc-postgres-cdc:5432/nyc_taxi	5000 dòng random_st
cdc_register	nyc-pipeline-tools:latest	entrypoint-cdc-register --debezium-url http://svc-debezium:8083 --postgres-host svc-postgres-cdc	Idempotent
cdc_bridge	nyc-pipeline-tools:latest	entrypoint-cdc-bridge --bootstrap-server svc-kafka:9092 --input-topic nyc_cdc.public.trips --output-topic taxi.trip.events --idle-timeout 30 --flush-interval 500	--idle-ti 30s không is_delete

DAG này **không trigger** task Spark nào. Sau khi `cdc_bridge` thoát, các event nó produce nằm trong topic `taxi.trip.events` . Để consume phải chạy `spark_streaming` riêng (qua task trong E2E DAG hoặc `make spark-streaming`).

5.4 Hành vi chung và lưu ý

- Không task nào có `depends_on_past=True` . Có thể clear và re-run một task đơn lẻ mà không ảnh hưởng state của task downstream.
- `startup_timeout_seconds=600` chỉ set trên `trino_bootstrap` . Tất cả task khác dùng KPO default 120s.

- `is_delete_operator_pod` chỉ là `False` trên `trino_bootstrap` và `cdc_seed` (đều trong `nyc_e2e_pipeline`). Khi debug hai task này, pod sẽ tồn tại sau khi task xong; các task khác tự cleanup.
 - **ServiceAccount:** `airflow-sa` (định nghĩa trong `charts/nyc-taxi/templates/airflow/rbac.yaml`). Có quyền `get/list/watch` và `create/delete` trên pods và pod logs.
 - **Tham chiếu image** giả định image đã được load vào kind cluster local (Skaffold pre-deploy hook xử lý việc này). Nếu chạy trên cluster remote, đổi `image_pull_policy` thành `Always` và push image lên registry.
 - **Ngữ nghĩa restart:** task fail sẽ retry 3 lần (2 lần cho CDC) với backoff 30s. Sau 3 lần fail, task được đánh failed và DAG run fail; các task downstream bị skip.
-

6. Triển khai (Kubernetes / Skaffold — chính)

6.1 Một lệnh deploy

```
# Lần đầu:  
bash scripts/cluster_up.sh           # tạo kind cluster + load public images  
  
# Mỗi lần:  
skaffold dev --namespace nyc-taxi    # build images, deploy Helm, port-forward, watch
```

Skaffold:

- Build 4 image: `nyc-pipeline-tools` , `nyc-dbt` , `nyc-airflow` , `nyc-superset` .
- Pre-deploy hook: xoá immutable job trong namespace, tar-sync `airflow/dags/` , `jobs/` , `scripts/` , `dbt/` , `charts/` vào `project-files-pvc` hostPath trên node `kind-worker` .
- Helm deploy chart ở `charts/nyc-taxi/` (44 manifest K8s hợp lệ trong 13 folder thành phần).
- Port-forward 8 dịch vụ sang `localhost:39080-39087` .

6.2 Port-forward dịch vụ

Dịch vụ	URL	Thông tin đăng nhập
Superset	<code>http://localhost:39080</code>	<code>admin</code> / <code>admin</code>
MinIO API	<code>http://localhost:39081</code>	<code>minio</code> / <code>minio123</code>
Kafka UI	<code>http://localhost:39082</code>	—
Spark Master	<code>http://localhost:39083</code>	—
Trino	<code>http://localhost:39084</code>	—
Airflow	<code>http://localhost:39085</code>	<code>admin</code> / <code>admin</code>
MinIO Console	<code>http://localhost:39086</code>	<code>minio</code> / <code>minio123</code>
Postgres CDC	<code>http://localhost:39087</code>	<code>postgres</code> / <code>postgres</code>

6.3 Tên service K8s (bắt buộc prefix `svc-`)

Bên trong cluster, tất cả tên service đều có prefix `svc-` . Code trong repo dùng trực tiếp các tên này:

- `svc-minio:9000` (S3A + Trino Hive connector)
- `svc-trino:8080` (Trino JDBC)
- `svc-kafka:9092` (Spark streaming)
- `svc-postgres-cdc:5432` (Debezium + cdc_seed)
- `svc-postgres-analytics:5432` (materialize + Superset)
- `svc-debezium:8083` (cdc_register REST API)
- `svc-superset:8088` (Superset API)
- `svc-airflow-webserver:8080` (Airflow web UI)

6.4 Tài nguyên K8s do chart tạo

Kind	Số lượng	Ví dụ
Deployment	8	kafka-ui, superset, trino, deb, airflow-webserver, airflow-scheduler, spark-master, spark-worker
StatefulSet	4	zookeeper, kafka, postgres-cdc, postgres-analytics, airflow-postgres
Service	12	svc-* cho mỗi thành phần trên
Job	4	topic-init, postgres-init, minio-setup, airflow-init
ConfigMap	3	airflow endpoint, superset config, trino config
PersistentVolume / PVC	5	project-files-pv, raw-data-pv, minio-data, postgres-cdc, postgres-analytics, airflow-postgres
ServiceAccount / Role / RoleBinding	1 set	airflow-sa
Namespace	1	nyc-taxi

Ghi chú: chart source có thêm 3 file template ở `charts/nyc-taxi/templates/trino/charts/nyc-taxi/templates/trino/*.yaml`

mà Helm không load do đường dẫn nested lỗi. Chúng là bản trùng của template chính và có thể xoá.

6.5 Cấu hình Trino (hiện tại)

- `docker/trino/etc/jvm.config`: `-Xmx2G` (heap 2 GB, G1GC, region 32 MB).
- `docker/trino/etc/config.properties`: single-node coordinator, port 8080.
- `docker/trino/etc/catalog/hive.properties`: file-based metastore ở `/data/trino-metastore`, `hive.recursive-directories=true`, `hive.s3.endpoint=http://minio:9000` (Compose) hoặc `svc-minio:9000` (K8s), `hive.s3.path-style-access=true`, `hive.ssl.enabled=false`, `hive.non-managed-table-creates-enabled=true`, `hive.non-managed-table-writes-enabled=true`.

6.6 Cấu hình Postgres CDC (hiện tại)

- Image: `postgres:16-alpine`.
- Replication args: `wal_level=logical`, `max_replication_slots=4`, `max_wal_senders=4`.
- StatefulSet với PVC (không backup, không HA trong dev).
- `POSTGRES_PASSWORD=postgres`, `POSTGRES_USER=postgres`, `POSTGRES_DB=nyc_taxi`.

6.7 Cấu hình Kafka (hiện tại)

- Image: `confluentinc/cp-kafka` (7.x).
- Single broker, single pod.

- `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1` ,
`KAFKA_TRANSACTION_STATE_LOG_*_REPLICATION_FACTOR=1` .
 - Topics tạo khi khởi động bằng `scripts/create_kafka_topics.py` :
`taxi.trip.events` (3 partition), `taxi.trip.invalid` (3 partition),
`taxi.trip.dlq` (3 partition).
-

7. Phát triển local (Docker Compose)

```
make infra-up          # start ZK + Kafka + MinIO + Spark
make kafka-topics      # tạo topics
make spark-batch MONTH=03
make trino-bootstrap
make dbt-build
make gold-export
make superset-bootstrap
make verify-all       # E2E 7 bước
make clean-all
```

29 target Makefile tổng. Compose stack có 16 service, 6 profile group
(`default` , `tools` , `trino` , `dbt` , `superset` , `airflow`).

Cổng ở chế độ Compose (khác K8s):

Dịch vụ	Cổng
Superset	8088
MinIO Console	9001

Dịch vụ	Cổng
Airflow	8085
Kafka UI	8080
Spark Master	8081
Spark Worker	8082
Trino (JDBC)	8083

8. Validation

Sau một full pipeline run, các check sau được kỳ vọng pass:

Check	Cách chạy	Kỳ vọng
<code>dbt build</code>	<code>make dbt-build</code> hoặc DAG <code>nyc_analytics_refresh</code>	30 model, 27 test, 0 lỗi
Mart row counts	<code>make verify-mart</code> hoặc <code>scripts/verify_mart.py</code>	<code>dim_zone</code> $\approx$ 261, <code>fact_trips</code> $\approx$ 8.4M, <code>mart_hourly_summary</code> $\approx$ 11K, <code>mart_revenue_by_day</code> $\approx$ 90
Analytics questions	<code>make verify-analytics</code> hoặc <code>scripts/run_analytics_questions.py</code>	10/10 PASS (mỗi query trả $\geq$ 1 dòng)
CDC pipeline	<code>make verify-cdc</code>	Postgres row count > 0, Debezium connector RUNNING
Anomaly check	<code>scripts/check_anomaly.py</code> (DAG task)	chỉ thông tin, exit 0

`run_analytics_questions.py` đọc `sql/analytics_questions.sql`, tách theo marker title `-- N)`, chạy mỗi câu trên Trino, và assert mỗi câu

trả về ít nhất một dòng. Mỗi câu phải kết thúc bằng `;`.

9. Giới hạn đã biết

Đây là đặc điểm của code tính đến thời điểm viết README; project là demo, không phải platform production-ready.

9.1 Chuỗi lỗi validation khác nhau giữa batch và streaming

Cùng một rule logic nhưng sinh chuỗi lỗi khác nhau trong

`jobs/spark_local_batch.py` (vd `non_positive_trip_distance`) so với `jobs/spark_stream_taxi_events.py` (vd `trip_distance_must_be_gt_0`).

5 rule bị lệch. Một cleanup tương lai nên align và quyết định một chuẩn đặt tên.

9.2 Batch và streaming ghi cùng đường dẫn silver

Cả hai job ghi vào `s3a://nyc-silver/trips/`. Không có cột hay metadata đánh dấu nguồn dòng. Điều này khiến không thể monitor độc lập output batch và stream, hoặc rollback một phần mà không ảnh hưởng phần còn lại.

9.3 CDC bridge sinh consumer group mới mỗi run

`scripts/cdc_bridge.py` dùng

`group_id = f"cdc-bridge-{uuid4().hex[:8]}"`, nên mỗi lần gọi bắt đầu từ offset sớm nhất. Chấp nhận được cho one-shot DAG (bridge thoát sau

`--idle-timeout` giây), nhưng gây trùng event nếu bridge bị restart trong khi topic nguồn còn data chưa consume.

9.4 Không có quality gate trong main DAG

Không có task `verify_silver` / `verify_gold` / `verify_postgres` / `verify_superset` / `verify_freshness` trong bất kỳ DAG nào. `analytics_check` và `anomaly_check` chỉ thông tin (exit 0 luôn).

9.5 Trino chạy single node với file-based metastore

- Single coordinator (không HA).
- Metastore: file-based tại `/opt/project/data/trino-metastore` (PVC). Nếu PVC mất, catalog phải tạo lại bằng cách chạy lại `scripts/trino_register.py`.
- JVM heap: `-Xmx2G` — query CTAS lớn có thể OOM.
- `gold_export` chạy 33 CTAS tuần tự. Bước `clean_s3_path` đã có để recover khi fail một phần, nhưng một lần OOM vẫn có thể buộc retry toàn bộ task.

9.6 Helm chart có duplicate template lồng nhau

`charts/nyc-taxi/templates/trino/charts/nyc-taxi/templates/trino/` chứa 3 file trùng (configmap, deployment, service) mà Helm không load do đường dẫn. Có thể xoá; template thật ở `charts/nyc-taxi/templates/trino/*.yaml`.

9.7 Spark streaming dùng `failOnDataLoss=false`

Nếu Kafka retention drop một message giữa hai lần Spark run, việc skip im lặng che giấu data loss. Production nên chuyển sang `true` và vận hành topic có retention dài hơn.

9.8 Không monitor chuỗi CDC

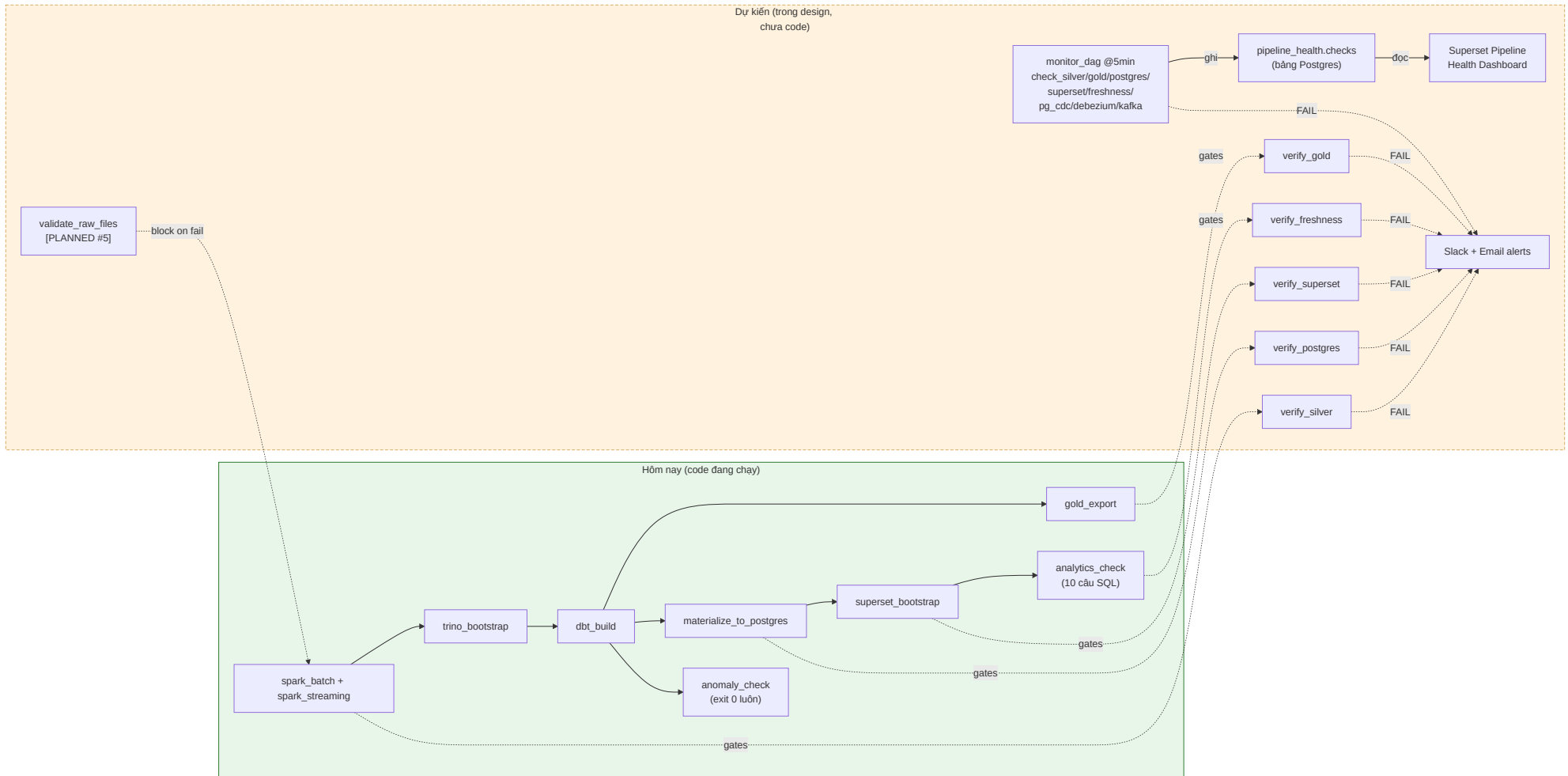
Không có DAG hay task nào poll trạng thái Debezium connector, Kafka consumer lag, hay Postgres replication slot lag. Lỗi trong chuỗi CDC chỉ phát hiện gián tiếp qua `anomaly_check` hoặc khi inspect `gold_dq_row_count_trend`.

10. Roadmap / dự kiến (chưa implement)

Phần này mô tả những gì design draft (`docs_update.md` + `new_doc.md`) đề xuất nhưng **chưa có trong code hiện tại**. Mỗi mục nên được implement trong một PR riêng với test riêng.

10.1 Hiện tại vs dự kiến

Pipeline hiện tại chạy như một DAG đơn, gần như không monitor. Quality gate và continuous monitoring đã được document trong design draft nhưng chưa implement.



Chú thích:

- **Nét liền** = code thực sự đang chạy hôm nay.
- **Nét đứt** = chưa implement; design draft đề xuất.
- **Hộp xanh** = main DAG hiện tại (không có quality gate, không có continuous monitor).
- **Hộp cam** = các thành phần dự kiến (5 verify gate + 1 monitor DAG)

- - 1 health table + 1 health dashboard + 1 alert channel).

10.2 Năm quality gate (dự kiến)

Main pipeline
(nyc_e2e_pipeline)

spark_batch

spark_streaming

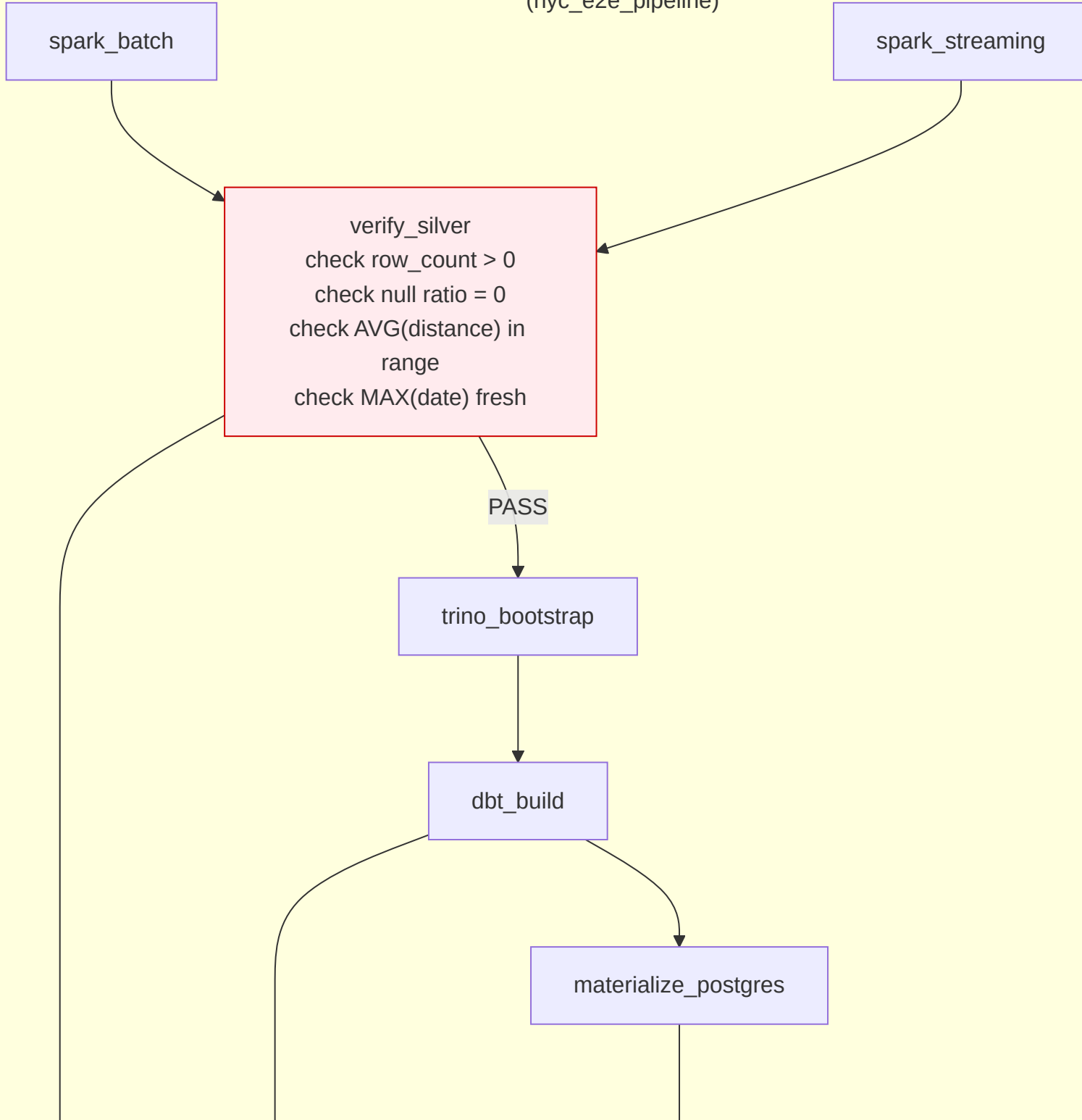
verify_silver
check row_count > 0
check null ratio = 0
check AVG(distance) in
range
check MAX(date) fresh

PASS

trino_bootstrap

dbt_build

materialize_postgres



FAIL

verify_postgres
check pg rows = gold rows
check all 33 tables match

PASS

superset_bootstrap

verify_superset
check charts render OK
check metrics match Trino

FAIL

Slack warning only

FAIL

PASS

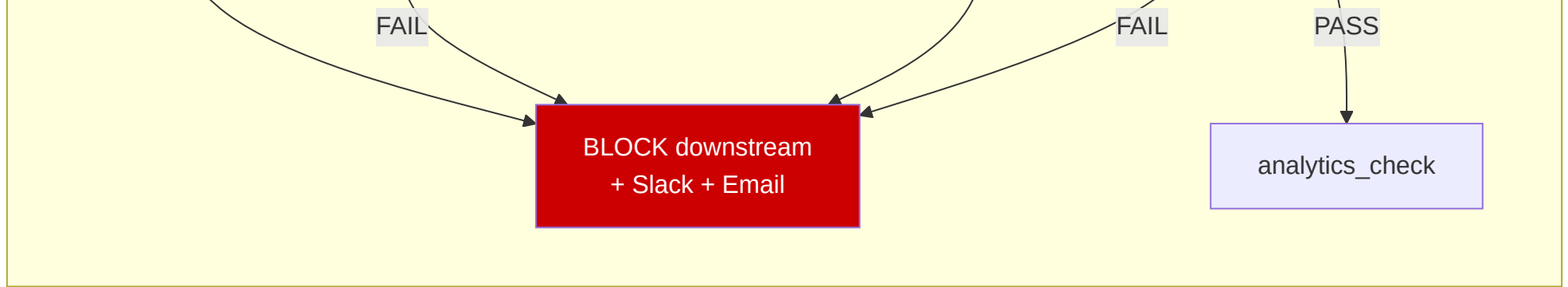
superset_saved_queries

verify_freshness
check MAX(date) le 35d
check rows in 7d range

verify_gold
check 30/30 tables exist
check row_count > 0
check match dbt source

gold_export

FAIL

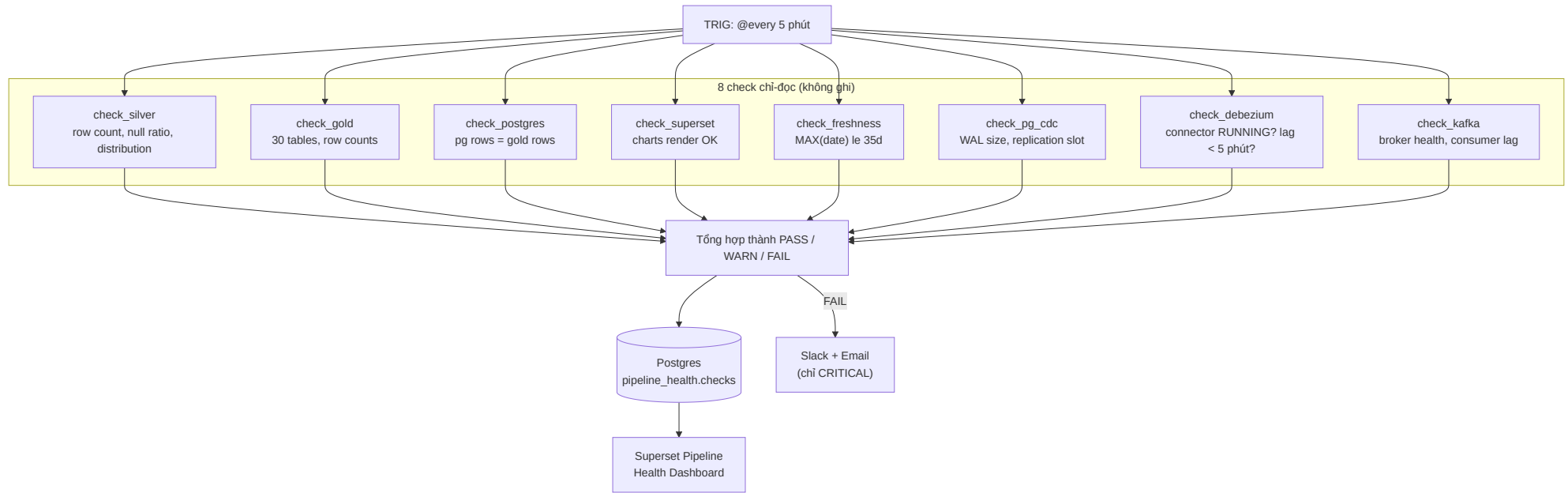


Ngữ nghĩa gate:

Gate	Đọc từ	Block ai	Kích hoạt bởi	Slack khi fail
verify_silver	Trino hive.nyc.trips	trino_bootstrap	mỗi DAG run	có + Email
verify_gold	Trino hive.nyc_gold.*	(chưa có)	mỗi DAG run	có + Email
verify_postgres	Postgres nyc_analytics.public.*	superset_bootstrap	mỗi DAG run	có + Email
verify_superset	Superset API	(không — Superset là bước cuối)	mỗi DAG run	có (warning)
verify_freshness	Trino hive.mart.fact_trips	analytics_check	mỗi DAG run	có + Email

verify_superset là warning thay vì block vì dashboard là bước cuối trong pipeline — không có downstream để block.

10.3 Monitor DAG (dự kiến)

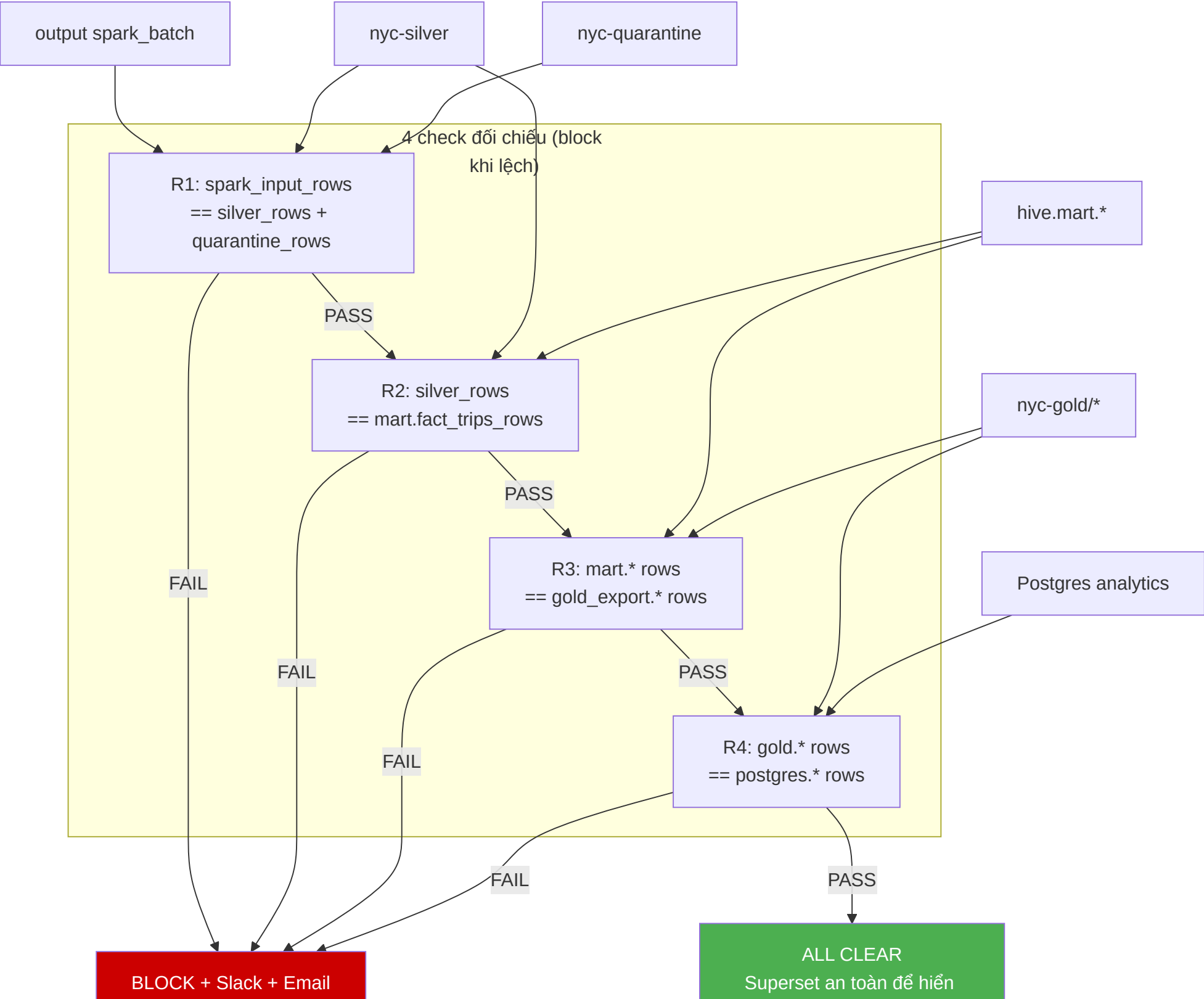


Đặc tính chính:

- **Chỉ đọc** — không bao giờ ghi vào data lake hay catalog. An toàn
- chạy song song với main pipeline.
- **Độc lập** — Monitor DAG fail không block hay ảnh hưởng main pipeline.
- Airflow coi hai DAG là các run độc lập.
- **Nhanh** — Mỗi check là một SELECT hoặc REST call. Tổng thời gian
- dự kiến dưới 30 giây.
- **Chi tiết** — Mỗi check có severity riêng (PASS / WARN / FAIL) và

- alert channel riêng.

10.4 Cross-node reconciliation (dự kiến)



Mỗi `R*` chạy như một task KubernetesPodOperator giữa các task pipeline tương ứng. Row count so sánh qua `SELECT COUNT(*)` mỗi bên; tolerance = 0 (khớp chính xác).

10.5 Roadmap 16 mục (chi tiết)

16 mục dưới đây nhóm theo layer pipeline. Effort ước tính cho một developer làm part-time.

| # | Tính năng | Layer | Nguồn | Effort |

+|---|---|---|---|---|

|| **Quality gates (nhóm A)** || ||

| 1 | **Quality gates** — 5 inline task `verify_*` (`verify_silver`, `verify_gold`, `verify_postgres`, `verify_superset`, `verify_freshness`) block DAG khi fail | main DAG | `docs_update.md` §2 | 1 tuần |

| 3 | **Cross-node reconciliation** — 4 check row count (`spark_input == silver + quarantine`, `silver == mart.fact_trips`, `mart. == gold.`, `gold. == postgres.`) | main DAG | `docs_update.md` §4 | 3 ngày |

| 13 | **Output contracts YAML** — contract khai báo per-node (`contracts/silver.yaml`, `contracts/gold.yaml`, ...) được verify task dùng | main DAG | `AGENTS.md` Output Contracts | 3 ngày |

|| **Monitoring (nhóm B)** || ||

| 2 | **Monitor DAG** — DAG riêng chạy mỗi 5 phút với 8 check chỉ-đọc (`silver` / `gold` / `postgres` / `superset` / `freshness` / `pg_cdc` / `debezium` / `kafka`) → Slack + Email khi fail | monitor DAG | `docs_update.md` §2 | 1 tuần |

| 4 | **Pipeline Health Dashboard** — Superset dashboard trên bảng `pipeline_health.checks` được monitor DAG populate mỗi 5 phút | monitor DAG | `AGENTS.md` Health Dashboard section | 1 tuần |

| 12 | **Alert integration** — Slack + Email khi DAG fail, OOM, anomaly, freshness violation | monitor DAG | `AGENTS.md`

Failure Mode Coverage + `docs/15-dataops-roadmap.md` | 1 tuần |

|| **Data integrity (nhóm C)** || ||

| 5 | **MinIO pre-ingest validation** — `scripts/validate_raw_files.py` quarantine Parquet hỏng trước khi `spark_batch`

chạy | ingest | [docs_update.md](#) §12 | 3 ngày |

| 14 | **Spark batch crash recovery** — ghi vào `_tmp/` rồi atomic move sang `silver/trips/` khi thành công; kiểm row count sau retry | ingest | [docs_update.md](#) Spark Scalability section | 3 ngày |

| 15 | **Spark batch auto-split theo tháng** — khi pod < 8 GB RAM, tách thành 12 sub-task tuần tự thay vì một `local[*]` lớn | ingest | [docs_update.md](#) Spark Scalability section | 3 ngày |

|| **CDC chain (nhóm D)** ||||

| 6 | **CDC chain hardening** — Postgres WAL size + slot monitor, Debezium `snapshot.mode=schema_only` + `delete.handling.mode=rewrite`, Kafka idempotent producer + `enable.idempotence` + retention dài hơn + DLQ, cdc_bridge group_id cố định + `auto.offset.reset=latest` | CDC | [docs_update.md](#) §8–11 | 1 tuần |

|| **Compute (nhóm E)** ||||

| 7 | **Trino resource groups** — `gold_export` cap 2 concurrent × 3 GB, `adhoc` 3 × 2 GB; batched CTAS (3 × 10) với pause 30s giữa batch | catalog | [docs_update.md](#) §13 | 3 ngày |

| 8 | **dbt CI + incremental models + business tests** — GitHub Actions chạy `dbt build --target staging` trên PR, `fact_trips` `materialized='incremental'`, cross-model revenue assertions | transform | [docs_update.md](#) §14 | 1 tuần |

|| **Presentation (nhóm F)** ||||

| 9 | **Superset hardening** — idempotent bootstrap (đã gần đúng), cache-bust API sau pipeline, chart SQL version control trong `superset/charts/`, secret rotation qua env var, dashboard export sang git | serve | [docs_update.md](#) §15 | 3 ngày |

| 10 | **Anomaly check upgrade** — multi-metric (fare / distance / revenue), 30-day rolling $\pm 3\sigma$ baseline, optional `--block` flag, Slack webhook | serve | [docs_update.md](#) §16 | 2 ngày |

|| **Infrastructure (nhóm G)** ||||

| 11 | **Staging environment** — namespace + MinIO bucket + Postgres riêng cho pre-merge testing | infra | [docs/15-dataops-roadmap.md](#) | 2 tuần |

| 16 | **Production deployment** — Trino 1 coord + 2 workers, Kafka 3 brokers, Postgres CDC trên RDS Multi-AZ, MinIO → S3, Spark trên EMR/Glue | infra | [new_doc.md](#) Production Hardening + [docs_update.md](#) §8 | 2+ tuần |

Tổng effort (cộng theo nhóm):

+ - Nhóm A (gate + contract): ~3 tuần

+ - Nhóm B (monitor + dashboard + alert): ~3 tuần

- + - Nhóm C (hardening ingest): ~1.5 tuần
- + - Nhóm D (chuỗi CDC): ~1 tuần
- + - Nhóm E (Trino + dbt): ~2 tuần
- + - Nhóm F (Superset + anomaly): ~1 tuần
- + - Nhóm G (infra + production): ~4+ tuần

- **Tổng: ~15 tuần** cho một developer part-time

10.6 Gap đã biết mà roadmap chưa cover

Đây là issue đang mở theo dõi trong `docs/issues.md` nhưng chưa lên lịch. Chúng nằm ngoài 16 mục trên và cần thảo luận riêng:

- + - `is_delete_operator_pod=False` chỉ set trên 2/13 DAG task
 - (`trino_bootstrap` và `cdc_seed`); 11 task còn lại vẫn drop pod log
 - khi xong. (§5.4 liệt kê task nào.)
 - CDC bridge's random `group_id` gây trùng event khi restart
 - (xem §9.3). Mục #6 fix một phần nhưng không phải lựa chọn
 - `group_id` bản thân nó.
 - Streaming và batch chia sẻ đường dẫn silver đơn lẻ, không có source
 - marker (xem §9.2). Roadmap đề xuất tách `nyc-silver/batch/trips` và `nyc-silver/stream/trips`, nhưng chỉ như một phần của mục #6
 - (CDC hardening) và chưa chính thức hoá.
 - `scripts/check_anomaly.py` luôn trả exit 0 (chỉ thông tin), nên
 - không thể block DAG dù design muốn vậy. Mục #10 đề xuất flag `--block`, nhưng code hiện tại cần wire-up trước.

- Tên chuỗi lỗi validation khác nhau giữa batch và streaming
- (xem §9.1). Roadmap không đề cập.

11. Thuật ngữ

Thuật ngữ	Nghĩa
silver	Tầng "trusted" — mỗi dòng đã pass 10 (hoặc 11) validation rule. Partition theo <code>pickup_year/pickup_month</code> .
quarantine	Tầng "rejected" — cùng schema với silver cộng <code>validation_errors</code> <code>ARRAY<VARCHAR></code> và <code>quarantine_ts</code> . Không partition.
gold	Tầng "presentation". 33 dataset Parquet sinh ra bởi <code>export_gold_to_minio.py</code> từ Trino views.
mart	Tầng trung gian aggregate trong dbt (8 views).
staging	Tầng dbt 1:1 cast-and-rename trên các bảng external thô (3 views).
validators	10 business rule trong <code>spark_local_batch.py</code> và bộ song song trong <code>spark_stream_taxi_events.py</code> .
trip_id	<code>`xxhash64(concat_ws(</code>

12. License & nguồn dữ liệu

- Code pipeline: xem project root cho license.
- Nguồn dữ liệu: [NYC TLC Yellow Taxi trip records](#).
- Dataset test đính kèm trong repo: 3 tháng đầu 2024 (tháng 1 / 2 / 3), ~8.4M valid + 1.07M invalid dòng.